GARMENT INTELLIGENCE: AN INTEGRATED VISION BASED PLATFORM FOR PRODUCTION MONITORING, QUALITY CHECKING AND PACKING DECISION SUPPORT IN APPAREL MANUFACTURING

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
Dharmarathna P.W.A.N.	IT21255892	C1 Production Monitoring: vision based garment piece counting and sewing cycle time measurement
Pathiraja H.P.M.H.D.S.	IT22627582	C2 Rework Counter: real time good and rework counting at the quality checking workstation
Jayaweera M.P.Y.C.	IT22556370	C3 Garment Style and Colour Detection with production decision support
Jayarathna W.G.U.C.G.	IT22187796	C4 Garment Size Detection: real time best frame selection and marker free size measurement

Group ID: R26 IT 054

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

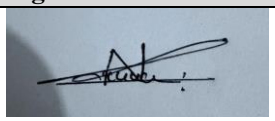
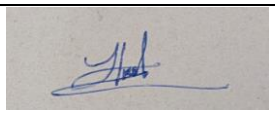
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Dharmarathna P.W.A.N.	IT21255892		11/10/2026
Pathiraja H.P.M.H.D.S.	IT22627582		11/10/2026
Jayaweera M.P.Y.C.	IT22556370		11/10/2026
Jayarathna W.G.U.C.G.	IT22187796		11/10/2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Narmada Gamage		
Co-supervisor: Ms. Buddhima Attanayaka		

ABSTRACT

Apparel factories still learn what happened on the line from hand tallies that reach supervisors hours later: how many pieces each sewing workstation completed, how many garments failed inspection, and how many of each style, colour and size were packed. This report presents Garment Intelligence, the integrated solution of research group R26-IT-054, which replaces these tallies with camera based measurement at three production stages and brings the results together in one offline desktop platform. Four components address four subproblems. Production Monitoring counts completed sewing operations and measures cycle time with a YOLOv8n workstation gate, an eight frame MobileNetV3-Small state classifier and a completion decoder, with rework and downtime entered on a Bluetooth controller. The Rework Counter records each accept and reject decision at the quality checking table from the direction in which the inspector moves the garment. Style and Colour Detection recognises four garment styles, measures their colours and turns packed counts into downtime aware delivery advice. Garment Size Detection selects suitable frames, segments the garment under a calibrated camera, measures it in centimetres and assigns a chart size. An Electron shell starts each component as an isolated local service with its own model and database. The components reached, on held out or validation data, 98.10 % state accuracy (C1), mAP@0.5 of 0.938 (C2), 0.958 (C3) and a mask mAP@0.5 of 0.960 (C4), with width and length errors of 0.570 cm and 0.674 cm against a tape. For this report, 188 automated integration and service tests were executed and passed, alongside 107 reported for C3. The work shows that ordinary webcams and lightweight models can supply timely production records across the line, and that reliability depends as much on the event logic and integration boundaries as on the models. Field accuracy against independent counts remains the main open validation step.

Keywords: apparel manufacturing, production monitoring, computer vision, quality checking, integrated desktop platform

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor, Ms. Narmada Gamage, and our co supervisor, Ms. Buddhima Attanayaka, for their guidance and encouragement throughout this project. Their insistence that every figure we report be traceable to evidence shaped both our individual work and the way this report evaluates the integrated solution.

We thank the sewing operators, inspectors and workplace representatives who allowed us to observe and record their work. Seeing how pieces are counted, checked and packed on real lines gave the project its purpose, and the recordings they permitted made the models possible.

We are grateful to the academic and administrative staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for the knowledge and facilities provided during our degree, and to our families and friends for their patience and support.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1
1.1 Background and context 1
1.2 Stakeholders and user needs 1
1.3 Alignment with the Sustainable Development Goals 1
1.4 Purpose and scope of the common report 1
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 1
2.1 Overall problem definition 1
2.2 Root causes and overall research gap 1
2.3 Subproblem and component mapping 1
2.4 Overall aim and group objectives 1
2.5 Scope assumptions and constraints 1
3 OVERALL SOLUTION STRUCTURE 1
3.1 Integrated solution overview 1
3.2 High level architecture 1
3.3 Component 1 summary 1
3.4 Component 2 summary 1
3.5 Component 3 summary 1
3.6 Component 4 summary 1
3.7 End to end user and data flow 1
4 COMPONENT INTEGRATION AND COHESIVE OPERATION 1
4.1 Integration strategy 1
4.2 Interfaces data contracts and shared models 1
4.3 Shared nonfunctional requirements 1
4.4 Integrated workflow and exception handling 1
4.5 Team contribution and dependency matrix 1
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 1
5.1 Common technology stack and infrastructure 1
5.2 Development and integration process 1
5.3 Deployment and operational configuration 1
5.4 Evidence of team execution 1
6 INTEGRATED TESTING EVALUATION AND VALIDATION 1
6.1 Evaluation framework and success criteria 1
6.2 Integration and end to end testing 1

6.3 Overall results 1
6.4 Stakeholder and user validation 1
6.5 Achievement of common objectives 1
6.6 Integrated limitations and threats 1
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 1
7.1 Value proposition and beneficiaries 1
7.2 Market alternatives and differentiation 1
7.3 Business model and feasibility 1
7.4 Adoption intellectual property and compliance 1
7.5 Sustainable Development Goal contribution 1
8 CONCLUSION AND FUTURE DIRECTIONS 1
8.1 Overall conclusion 1
8.2 Principal integrated contribution 1
8.3 Future directions 1
REFERENCES 1
APPENDICES 1
Appendix A Integrated interface specification 1
Appendix B Team contribution declaration 1
Appendix C Academic integrity and AI use disclosure 1
Appendix D Repository commit graph 1

LIST OF FIGURES

Figure 1.1: Position of the four research components in the garment production flow..... 1

Figure 3.1: High level architecture of the Garment Intelligence platform 1

Figure 3.2: Component 1 workstation preparation screen in the integrated application..... 1

Figure 3.3: Component 2 live production workspace with input source, zone settings and live counts 1

Figure 3.4: Component 3 camera preparation screen 1

Figure 3.5: Component 3 production decision support dashboard for the packing zone..... 1

Figure 3.6: Measurement pipeline of Component 4, Garment Size Detection 1

Figure 3.7: Component 4 packing production dashboard during a live session .. 1

Figure 3.8: Landing screen of the integrated platform..... 1

Figure 3.9: Workstation selection screen with one card for each component 1

Figure 3.10: Operator flow through the platform..... 1

Figure 5.1: Microsoft Planner plan for Component 3 in the R26-IT-054 Teams workspace..... 1

Figure 5.2: Commits per week to the shared repository over the project year 1

Figure 5.3: Weekly commits to the main branch in the last three months, excluding merge commits 1

Figure 5.4: Commits by contributor to the main branch in the last three months 1

Figure 6.1: Automated test results by suite..... 1

Figure 6.2: Absolute width and length errors of Component 4 in five tape comparisons 1

Figure D.1: Commit graph, 4 May to 21 July 2026: shared frontend, first screens and C1 dataset preparation 1

Figure D.2: Commit graph, 21 July to 21 August 2026: C1 classifier training, C3 model and pipeline, C4 best frame selection and size chart..... 1

Figure D.3: Commit graph, 26 to 29 August 2026: C1 service, controller and decoder, C3 data service and C4 counting..... 1

Figure D.4: Commit graph, 30 August to 4 September 2026: local desktop application and size feature merge 1

Figure D.5: Commit graph, 13 September to 10 October 2026: C1 operation type model and final integration merges..... 1

LIST OF TABLES

Table 1.1: Stakeholders and their needs 1
Table 1.2: Alignment with the Sustainable Development Goals 1
Table 2.1: Subproblems and their components 1
Table 2.2: Group objectives, contributing components and evaluation evidence 1
Table 3.1: Summary of Component 1, Production Monitoring 1
Table 3.2: Summary of Component 2, Rework Counter 1
Table 3.3: Summary of Component 3, Garment Style and Colour Detection 1
Table 3.4: Summary of Component 4, Garment Size Detection 1
Table 4.1: Integration patterns used for the four components 1
Table 4.2: Shared contract between the platform shell and each component 1
Table 4.3: Shared nonfunctional requirements and how they are addressed 1
Table 4.4: Failure paths and recovery behaviour 1
Table 4.5: Team contribution matrix 1
Table 4.6: Dependency matrix (a mark means the row depends on the column) 1
Table 5.1: Common technology stack 1
Table 5.2: Trained models used by the integrated platform 1
Table 6.1: Group evaluation questions, success criteria and evidence 1
Table 6.2: Automated test suites of the integrated solution 1
Table 6.3: Headline results of the four components 1
Table 6.4: Achievement of the group objectives 1
Table 7.1: Value of the integrated platform for each beneficiary 1
Table A.1: Service configuration and endpoints of each component 1
Table A.2: IPC channels exposed to the renderer 1
Table B.1: Team contribution declaration 1

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BLE	Bluetooth Low Energy
CPU	Central Processing Unit
CSP	Content Security Policy
CSV	Comma Separated Values
FPY	First Pass Yield
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ID	Identifier
IoT	Internet of Things
IoU	Intersection over Union
IPC	Inter Process Communication
JSON	JavaScript Object Notation
mAP	mean Average Precision
MAE	Mean Absolute Error
MJPEG	Motion JPEG
OEE	Overall Equipment Effectiveness
QC	Quality Checking (Quality Control)
REST	Representational State Transfer
SDG	Sustainable Development Goal
SME	Small and Medium Enterprise
SQL	Structured Query Language
UI	User Interface
USB	Universal Serial Bus
YOLO	You Only Look Once

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Apparel manufacturing is one of Sri Lanka's most labour intensive industries and one of its leading export earners [1]. A garment order passes through cutting, sewing, quality checking and packing before it is shipped, and at every stage the factory needs the same kind of information: how much work has been completed, how much of it is good, and whether the order will finish on time. Linking physical work to timely digital information is a central idea of Industry 4.0 [2], yet in many plants this information is still produced by people with pens.

The four members of research group R26-IT-054 each observed one part of this problem. At sewing workstations, completed operations and their cycle times are counted by observation or reconstructed from shift totals. At the quality checking table, accepted and rejected garments are tallied on paper, so rework becomes visible only after it has cost time [3]. At the packing table, operators stop at intervals to count packed pieces, and a single total hides the mix of styles and colours in the order. When garments are sized for packing, measurements are taken by hand with a tape against a size chart. In each case the figure that reaches the supervisor is late, aggregated and difficult to check.

The common thread is that the work itself is visible. A camera above a workstation can see a garment being sewn, inspected, folded or laid flat. Modern object detectors run in real time on ordinary computers [4], [5], and lightweight classifiers can recognise activity in short video clips [6]. The group therefore set out to build one platform in which cameras and small models record production events at several stages of the line, and in which each stage's records are turned into information a supervisor can act on during the same shift. Figure 1.1 shows where the four research components sit in the flow of an order.

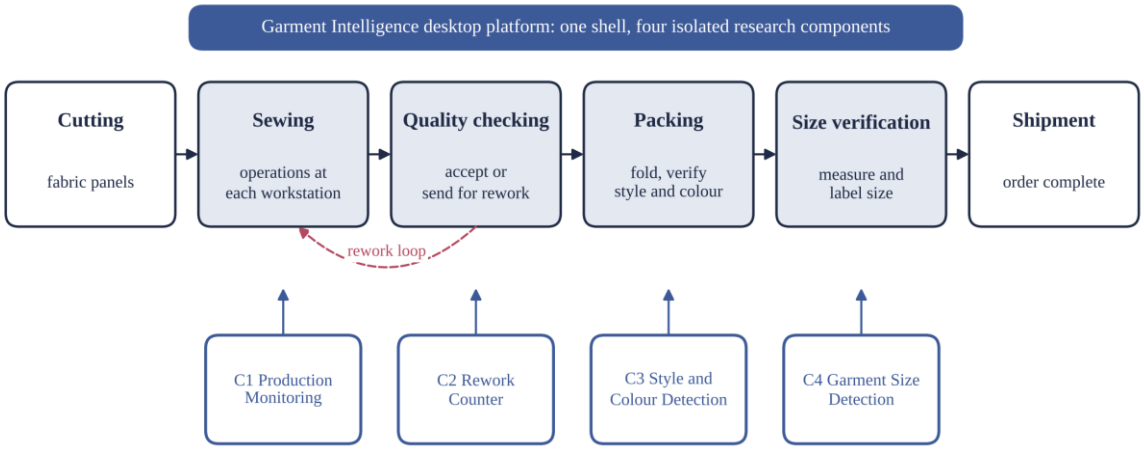


Figure 1.1: Position of the four research components in the garment production flow

1.2 Stakeholders and user needs

Table 1.1 identifies the stakeholders and the needs that shaped the solution. The needs were drawn from observation of sewing, checking and packing work during the project, from the recorded workstation videos that operators and workplace representatives allowed us to collect, and from the quality and production management literature [3], [7], [8].

Table 1.1: Stakeholders and their needs

Stakeholder	Role	Needs addressed by the solution
Sewing operator	Primary user (C1)	Counting without interrupting work; a simple way to signal rework and downtime
QC inspector	Primary user (C2)	Recording accept and reject decisions with no extra action
Packing operator	Primary user (C3, C4)	No tally sheet; clear on screen status; size verification without a tape
Line supervisor	Primary user (all)	Live counts, cycle time, pass rate, projected completion and overtime advice during the shift
Production planner	Secondary	Per style, colour and size progress against order targets; exports for planning
Quality manager	Secondary	Timed record of rework decisions and pass rate trends
Factory management and IT	Secondary	Low cost hardware, offline operation, data kept on site, simple installation
Buyers and auditors	Indirect	Traceable production and quality records; responsible use of cameras

Two needs were common to every group of users and became design principles for the whole platform. First, measurement must not add work: operators and inspectors should keep working as they do now, and the system should read their work rather

than ask them to report it. Second, every figure must be explainable: a supervisor who sees a count, a rate or a forecast must be able to trace it to the events that produced it, and the system must show its own uncertainty rather than hide it.

1.3 Alignment with the Sustainable Development Goals

The project is aligned with three of the United Nations Sustainable Development Goals [9], summarised in Table 1.2. The intended contribution is modest and indirect; the platform has not been deployed long enough at a working line to measure any impact, and the claims below describe the mechanisms through which a contribution would arise.

Table 1.2: Alignment with the Sustainable Development Goals

SDG	Relevant target area	Intended contribution of the platform
SDG 8 Decent work and economic growth	Productivity through technological upgrading; safe and fair work	Removes repetitive tallying; measures stations and orders rather than ranking individuals; makes overtime planned rather than last minute
SDG 9 Industry, innovation and infrastructure	Access of small and medium industries to technology	Brings vision based measurement to plants that cannot afford tagging or automated inspection, using webcams and existing computers
SDG 12 Responsible consumption and production	Reducing waste in production	Earlier visibility of rework, cycle time and order progress reduces repeated work, over production and wasteful rushes

1.4 Purpose and scope of the common report

This report presents the integrated problem and the integrated solution. It explains why the problem was divided into four subproblems, how the four components fit together as one platform, how they were integrated, how the complete solution was tested and evaluated, and what value it offers. The detailed research and development evidence for each component is given in the four individual theses [10], [11], [12], [13], and this report summarises rather than repeats it. Where a component's evidence is drawn from its thesis, the source is cited; where evidence was produced for this report, notably the execution of the integrated test suites and the inspection of the trained model checkpoints, the method is described here.

The components are numbered in the order in which a garment meets them: C1 Production Monitoring at the sewing workstation, C2 Rework Counter at the quality checking table, C3 Garment Style and Colour Detection at the packing table, and C4 Garment Size Detection at size verification before packing.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

Overall problem. In small and medium apparel factories, production and quality information at the sewing, quality checking and packing stages is recorded manually, at intervals and in aggregate, so supervisors receive it too late to act within the shift, cannot see it broken down by workstation, verdict, style, colour or size, and cannot verify it against the events that produced it.

The problem is deliberately stated without reference to a technology. Its three parts can each be observed directly: lateness, because tallies are totalled and transcribed after the work; aggregation, because one total hides the composition of the work; and lack of verifiability, because a written number carries no record of the events behind it. A solution is useful only if it improves all three without adding work at the stations.

2.2 Root causes and overall research gap

Four root causes lie behind the problem. Recording competes with production, so operators record as little and as rarely as possible; human inspection research shows that every recording task takes attention from the primary task [14]. The information needed at each stage is different in kind, so a single counting device does not fit all stations: a sewing workstation produces operations that have no fixed shape, a checking table produces decisions, and a packing table produces objects with attributes. Automatic alternatives are either intrusive, such as tags and scanners, or expensive and narrow, such as automated fabric inspection [15]. And measurement without interpretation does not support decisions: a count becomes useful only when set against time, targets, downtime and quality [8], [16].

Research addresses parts of this picture separately. Sewing work has been measured with machine current sensors [17] and sewing actions have been recognised in video [18]. Clothing detection and segmentation have strong benchmarks [19], [20]. Multi object trackers and line counters are mature [21]. But the reviewed work evaluates models on benchmark data rather than as instruments that turn station activity into

trustworthy production records, treats each stage in isolation, and rarely considers how several such instruments can be delivered, operated and trusted together on the ordinary hardware of a factory. The overall gap is therefore an integrated, low cost, camera based measurement layer that covers the stages where production and quality information is created, keeps each measurement explainable, and converts the records into decisions.

2.3 Subproblem and component mapping

The overall problem was divided into four subproblems, one per station type, shown in Table 2.1. They do not overlap, because each concerns a different physical event and a different station, but they are interdependent, because together they describe the progress and quality of the same order. Without C1 the platform cannot tell how fast the line produces; without C2 it cannot tell how much of that output must be produced again; without C3 and C4 it cannot tell what has actually been packed against the order's style, colour and size breakdown.

Table 2.1: Subproblems and their components

Subproblem	Station and event	Component and owner	Primary output
SP1 Completed sewing operations and their timing are not measured automatically	Sewing workstation; completion of an operation	C1 Production Monitoring; Dharmarathna P.W.A.N.	Piece count, completion timestamps, cycle time, rework and downtime events
SP2 Accept and reject decisions are tallied late and by hand	Quality checking table; inspector's decision	C2 Rework Counter; Pathiraja H.P.M.H.D.S.	Good and Rework verdicts per session, pass rate, estimated delay
SP3 Packed output is counted at intervals without style or colour breakdown	Packing table; packed garment	C3 Style and Colour Detection; Jayaweera M.P.Y.C.	Garment records with style and colours; efficiency, projected completion, overtime
SP4 Garment sizes are measured by hand against a chart	Size verification; flat garment	C4 Garment Size Detection; Jayarathna W.G.U.C.G.	Width and length in cm, size label, counts against order targets

2.4 Overall aim and group objectives

Aim. To design, implement and evaluate an integrated, offline desktop platform in which cameras and lightweight models measure production and quality events at the sewing, quality checking and packing stages of an apparel line, without adding work

for operators, and turn those measurements into explainable information for supervisors. Table 2.2 sets out the group objectives, the components that serve each one and the evidence used to evaluate it.

Table 2.2: Group objectives, contributing components and evaluation evidence

ID	Group objective	Components	Evaluation evidence
GO1	Measure production events automatically at three stages, without extra operator actions beyond normal work	C1, C2, C3, C4	Model evaluation, event or capture logic tests, replays and bench trials
GO2	Attach quality and attribute information to the events: cycle time, verdict, style, colour and size	C1, C2, C3, C4	Held out and validation metrics; logic and measurement tests
GO3	Turn records into decision information: pass rate, delay, efficiency, projected completion, overtime and order progress	C1, C2, C3, C4	Analytics tests and worked examples
GO4	Deliver the four components as one desktop platform with isolated, secure local services	Platform, all	Integration and security tests; integrated application runs
GO5	Run on ordinary factory hardware: USB webcams, an existing computer and a low cost controller	All	Hardware used in development and trials; CPU timing where measured
GO6	Verify the solution and report its limitations honestly	All	Automated test results; documented limitations

2.5 Scope assumptions and constraints

- **Stations.** One camera per station, mounted in a fixed position above the work area; one camera heavy feature is active on one computer at a time.
- **Offline operation.** Every component runs on the station computer; no cloud service is required and no video leaves the computer.
- **Human judgement.** Inspectors decide quality (C2) and operators signal rework and downtime (C1); the platform records these decisions rather than replacing them.
- **Classes and garment types.** C1 recognises two activity states; C2 detects one garment piece class; C3 recognises four styles (shirt, T-shirt, trouser, short); C4 measures three garment types (T-shirt, shirt, trouser) at a calibrated camera height of 100 cm.
- **Exclusions.** Defect classification, operator identification and ranking, cutting room measurement, and a factory wide server deployment are outside scope.

- **Dependencies.** Python 3.12 with PyTorch, Ultralytics and OpenCV for the services; Node.js and Electron for the desktop shell; an ESP32-C3 controller for C1's operator inputs.
- **Evaluation constraint.** The project period allowed model evaluation, automated testing, replays and short trials, but not multi week field trials against independent counts at a working line.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

Garment Intelligence is one Electron desktop application that brings the four research components together while preserving each member's frontend, backend, trained model and database. The operator opens the application, sees a landing screen that explains the platform, and chooses a workstation card: Rework Counter for the quality checking area, Production Monitoring for the sewing area, and Garment Style and Colour Detection or Garment Size Detection for the packing area. The platform then starts the service that card needs, loads its model, and opens that component's workspace. Leaving a workspace stops its camera so that the next feature can use it.

The central value proposition is a single measurement layer for the line. Each component turns a different kind of station activity into timestamped records with a clear meaning, keeps those records on the station computer, and presents them as decision information: cycle time and output for sewing, pass rate and delay for quality checking, efficiency, projected completion and overtime for packing, and order progress by size for size verification. All four follow the same principles: no added work for operators, evidence based acceptance of each record, visible uncertainty, and local processing.

3.2 High level architecture

Figure 3.1 shows the architecture. It has three layers. The renderer, written in React, contains the landing and selection screens and one workspace per component. The Electron main process holds all privileged functions: it exposes a small API to the renderer through a context isolated preload bridge [22], checks that every IPC message comes from the application's own window, registers two custom protocols that relay video streams, offers the Bluetooth device chooser, and supervises the four component services. Each component service is a separate Python process that listens only on 127.0.0.1, owns its camera, its trained model and its SQLite database, and never calls another component.

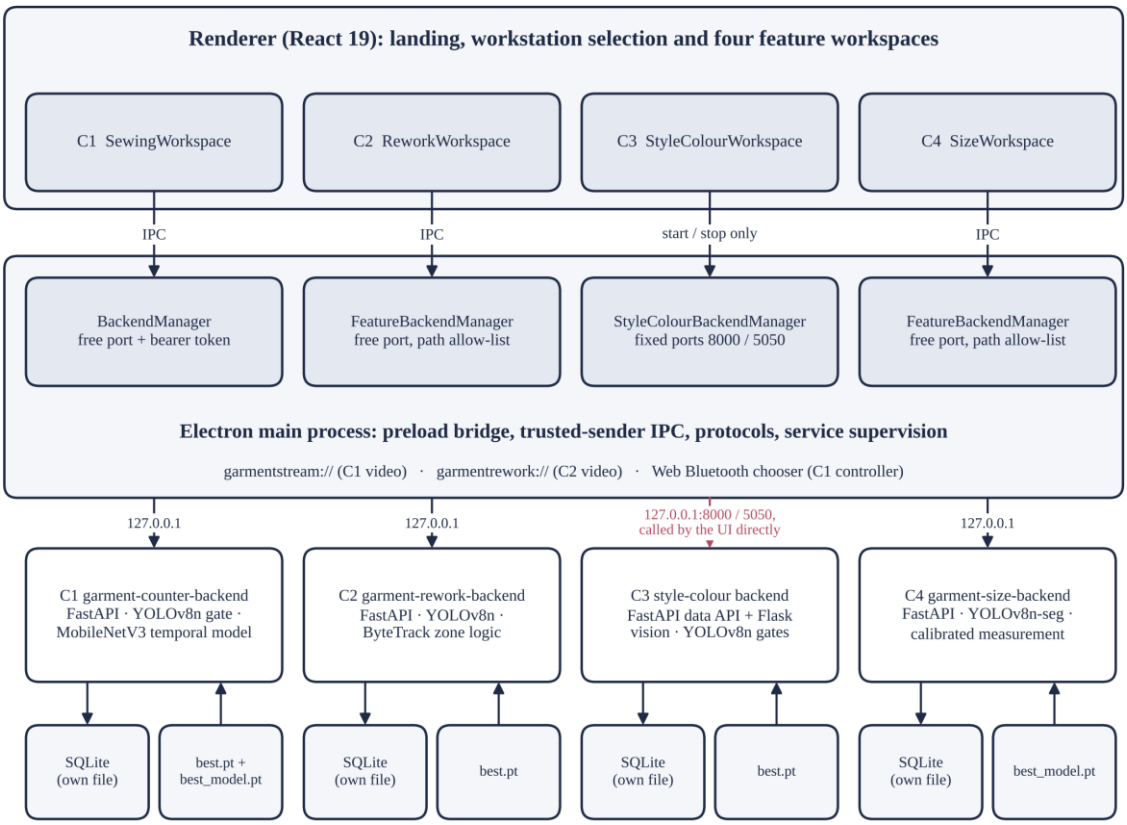


Figure 3.1: High level architecture of the Garment Intelligence platform

The architecture has two kinds of boundary. Vertical boundaries separate the components from each other: separate folders in the repository, separate processes at run time, separate models and separate databases under the user data directory, in features/garment-counter, features/rework, features/garment-style-colour and features/size. Horizontal boundaries separate the renderer, which handles presentation, from the main process, which handles privilege, and from the services, which handle cameras, models and data. External elements are the USB cameras used by all four components, the ESP32-C3 Bluetooth controller used by C1 [23], and the operating system's file dialogs used to choose workflow videos for testing.

3.3 Component 1 summary

Table 3.1: Summary of Component 1, Production Monitoring

Aspect	Summary
Owner	Dharmarathna P.W.A.N. (IT21255892) [10]
Subproblem	SP1: completed sewing operations and their cycle times are not measured automatically
Input	Fixed overhead camera at a sewing workstation; ESP32-C3 controller buttons for rework, downtime and reset
Core method	YOLOv8n workstation gate (confidence at least 0.50); eight frame, 1.5 second clips at 224 × 224 classified as Sewing or Idle/Setup by MobileNetV3-Small with temporal mean pooling; smoothing over five predictions; a decoder that arms after 2.0 s of stable Sewing, confirms after 0.6 s of stable Idle/Setup and applies a 3.0 s cooldown
Output	Confirmed piece count, completion timestamps and cycle times per session and employee; rework, downtime and reset events; analytics and Excel export
Contribution	Measures the rate at which the line produces, which every downstream figure depends on

C1 separates three questions that a single classifier would blur: whether the camera sees a valid workstation, what the operator is doing, and whether a production event has occurred [10], [24]. The state model was trained on 60 workstation videos with 907 annotated state segments and 438 manually marked completions, split by video into training, validation and test sets [10]. Rework and downtime are not inferred from video: the operator presses a button on a small Bluetooth controller, and the service records the event and pauses counting, so the model's visible state is never presented as a quality judgement.

Figure 3.2 shows the C1 workstation preparation screen in the integrated application. Before a session can be created or resumed, the operator selects and tests the sewing camera and connects the physical ESP32-C3 controller over Bluetooth, and a production preflight confirms that the camera has been tested, the AI models have been verified and the controller is connected. In the run shown, on a Windows machine with a Logitech C922 camera, the camera test passed and five of six checks were ready; the controller was not connected, so the screen kept the continue button disabled rather than allowing counting without a way to record rework and downtime.

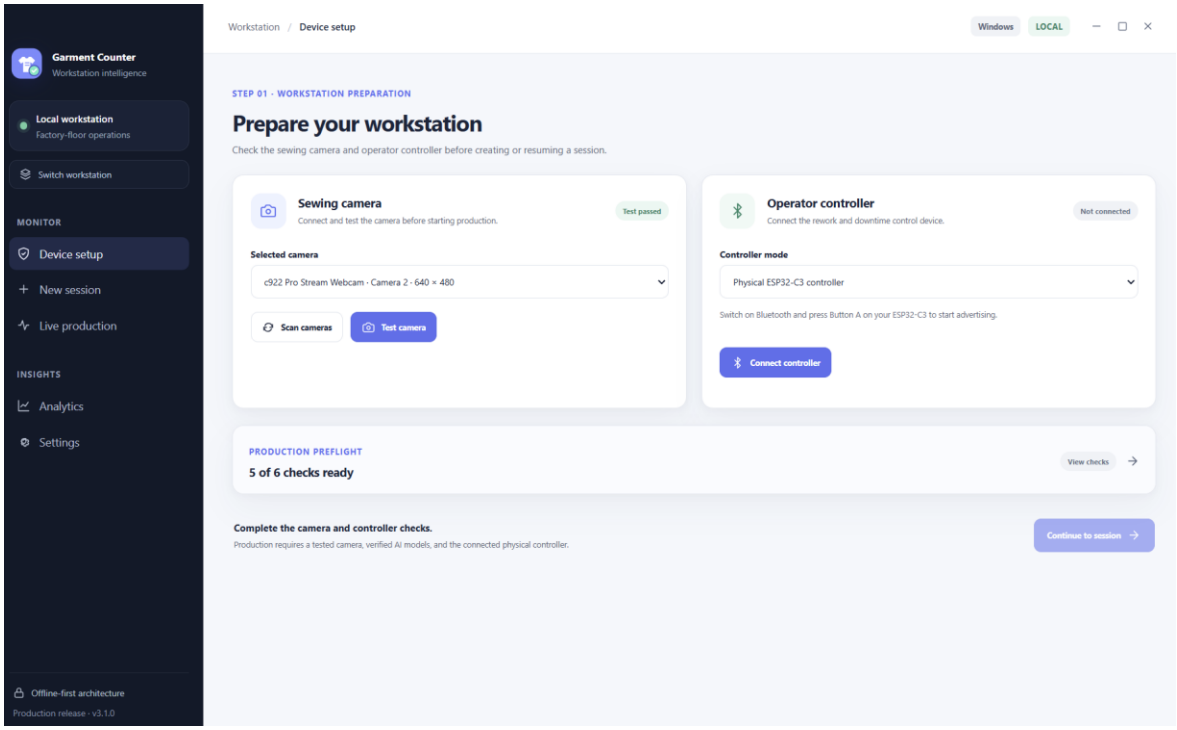


Figure 3.2: Component 1 workstation preparation screen in the integrated application

3.4 Component 2 summary

Table 3.2: Summary of Component 2, Rework Counter

Aspect	Summary
Owner	Pathiraja H.P.M.H.D.S. (IT22627582) [11]
Subproblem	SP2: accept and reject decisions at the quality checking table are tallied late and by hand
Input	Overhead camera above the checking table, or a workflow video for testing
Core method	Single class YOLOv8n garment piece detector; ByteTrack identities [21]; a two line zone rule that arms a track only after it is seen in the middle inspection zone and logs Good when it leaves above the top line or Rework when it leaves below the bottom line
Output	One verdict per garment with session and time; live Good and Rework counts, pass rate, estimated order delay, session history and CSV export
Contribution	Measures how much of the line's output must be produced again, without a defect classifier and without any extra action by the inspector

The key design decision in C2 is to keep the human inspector as the source of truth. The component reads the decision the inspector has already made from the direction in which the garment is placed. Its detector reached an mAP@0.5 of 0.938 on validation data, and a simulation that drove the shipped counting code with synthetic trajectories showed that the verdict logic is exact on clean tracks and identified which tracking imperfections cause which counting errors [11].

Figure 3.3 shows the C2 live production workspace. The inspector chooses a webcam or a workflow video as the input source and starts a session; the live counts of Good and Rework garments, the total checked, the pass rate and the estimated order delay appear beside the video. The advanced settings expose the detection sensitivity (0.25 in the run shown) and the positions of the top and bottom zone lines (30 % and 70 % of the frame height), and the order analytics take the average rework fix time (2 minutes) from which the delay is estimated.

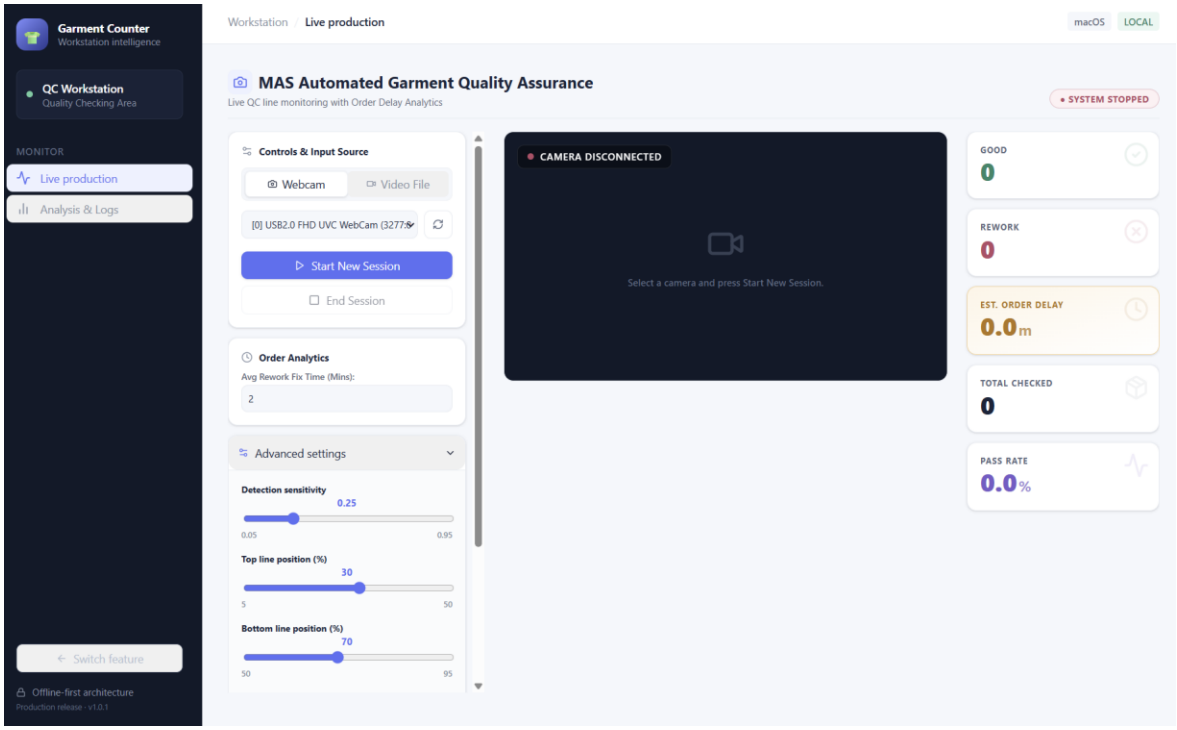


Figure 3.3: Component 2 live production workspace with input source, zone settings and live counts

3.5 Component 3 summary

Table 3.3: Summary of Component 3, Garment Style and Colour Detection

Aspect	Summary
Owner	Jayaweera M.P.Y.C. (IT22556370) [12]
Subproblem	SP3: packed output is counted at intervals and without style or colour breakdown
Input	USB camera above the folding zone of the packing table; order targets, shift and break settings; logged downtime
Core method	YOLOv8n detector for four styles (shirt, T-shirt, trouser, short) behind a cascade of six interpretable capture gates; colour measured from garment pixels using GrabCut [25], skin suppression and k means clustering in CIELAB; downtime aware production analytics
Output	One record per packed garment with style, up to five named colours, confidence and image; efficiency, projected completion, delay, overtime per day and a next day forecast, overall and per category
Contribution	Measures what has actually been packed against the order, and converts it into delivery and overtime advice

C3's research finding concerns the gap between benchmark accuracy and field reliability: a detector with test mAP@0.5 of 0.958 boxed whole frames on wide views and labelled a face as a shirt in live use, and the gate cascade closed these failures without retraining [12]. Its analytics separate downtime from pace by measuring output against effective working hours.

Figures 3.4 and 3.5 show C3 in the integrated application. The device setup screen guides the operator through choosing and testing the packing table camera before monitoring begins. The live dashboard then shows the total packed, the remaining quantity against the order target, the estimated remaining time, the projected delivery, the suggested overtime and the average cycle time based on effective packing time, followed by a breakdown for shirts, T-shirts, trousers and shorts. Until enough packing history exists, the projections are shown as awaiting data rather than as estimates.

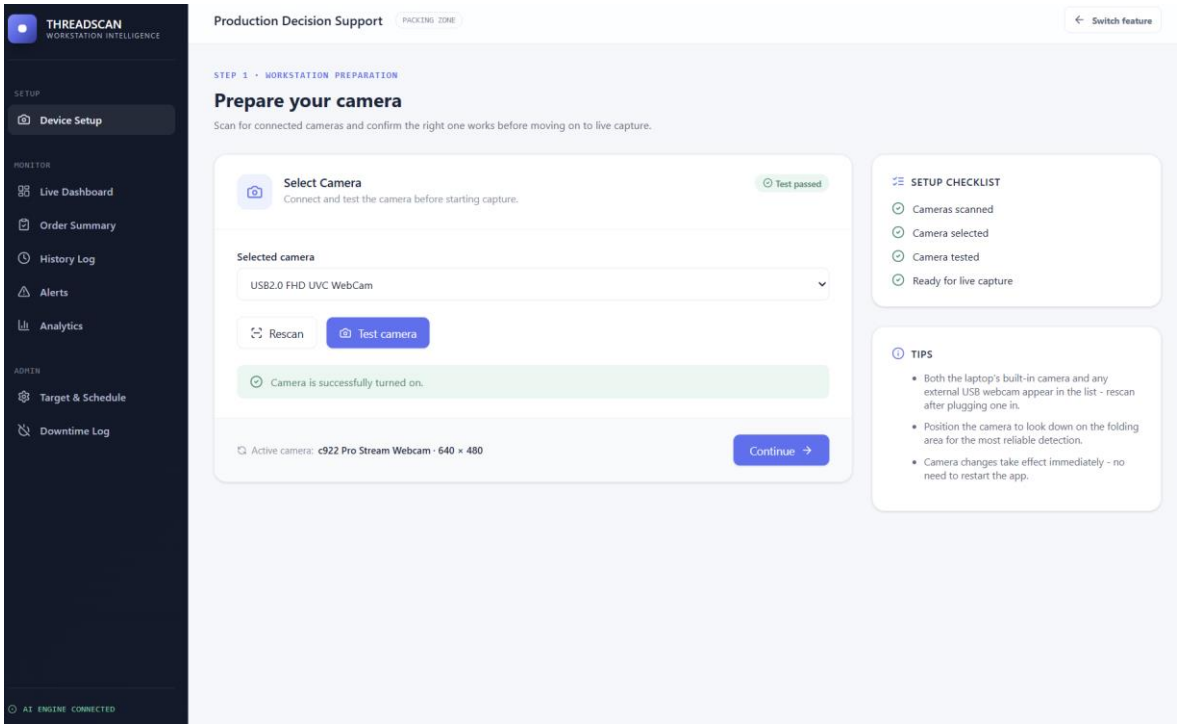


Figure 3.4: Component 3 camera preparation screen

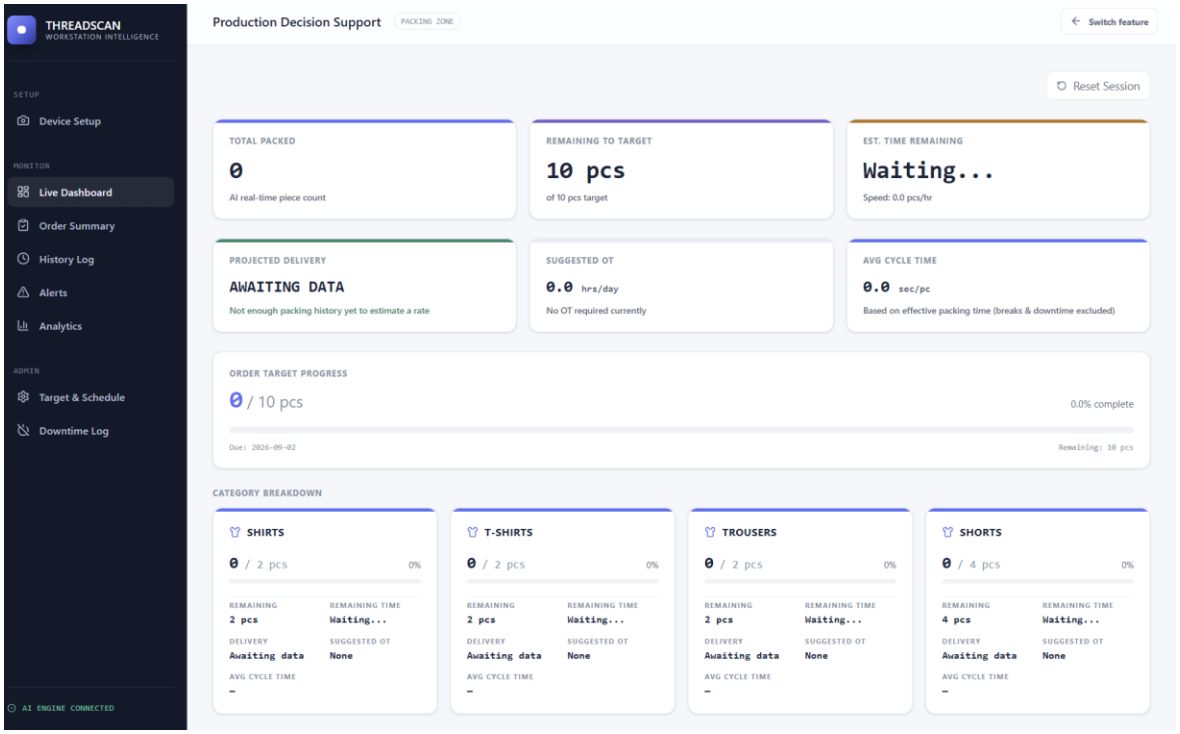


Figure 3.5: Component 3 production decision support dashboard for the packing zone

3.6 Component 4 summary

Table 3.4: Summary of Component 4, Garment Size Detection

Aspect	Summary
Owner	Jayarathna W.G.U.C.G. (IT22187796) [13]
Subproblem	SP4: garment sizes are verified by hand with a tape against a size chart
Input	Logitech C922 camera at a fixed height of 100 cm, perpendicular to the table, at 1920 × 1080; the garment type and order targets chosen by the operator
Core method	Frame suitability checks and temporal confirmation to select frames worth measuring; YOLOv8n-seg segmentation of a single garment class trained on 1,295 annotated images; mask refinement and deskewing; width and length measured along defined lines for each garment type; conversion to centimetres with a stored scale of 13.6 pixels per centimetre; nearest size in a local size chart
Output	Width, length, size label and confidence per garment; counts by type and size; production orders with S, M, L and XL targets, schedule, downtime, analytics and history
Contribution	Adds the size dimension of the order and replaces tape measurement for the supported garment types

C4 is framed in its thesis as real time best frame selection and marker free garment size measurement for smart apparel packing [13]: instead of measuring every frame, it measures only frames in which a complete, flat garment is clearly visible, and counts a garment only when several such frames agree. Garment footage was recorded at the 100 cm installation height, and selected frames were annotated with segmentation polygons in Roboflow; the final dataset (version 5) contains 1,295 images of one garment class, split into 1,070 training, 141 validation and 84 test images [13]. Figure 3.6 shows the measurement pipeline. The frontend captures a frame from the camera every 300 milliseconds and sends it to the service with the selected garment type. The service first checks that the frame has the calibrated resolution, since the scale of 13.6 pixels per centimetre is valid only for the 1920 × 1080 frame at 100 cm [26]. It segments the garment inside a zone inset by 4 % from each edge, retrying once on a contrast enhanced copy if no usable mask is found, which helps with dark garments on dark backgrounds and pale garments on pale ones.

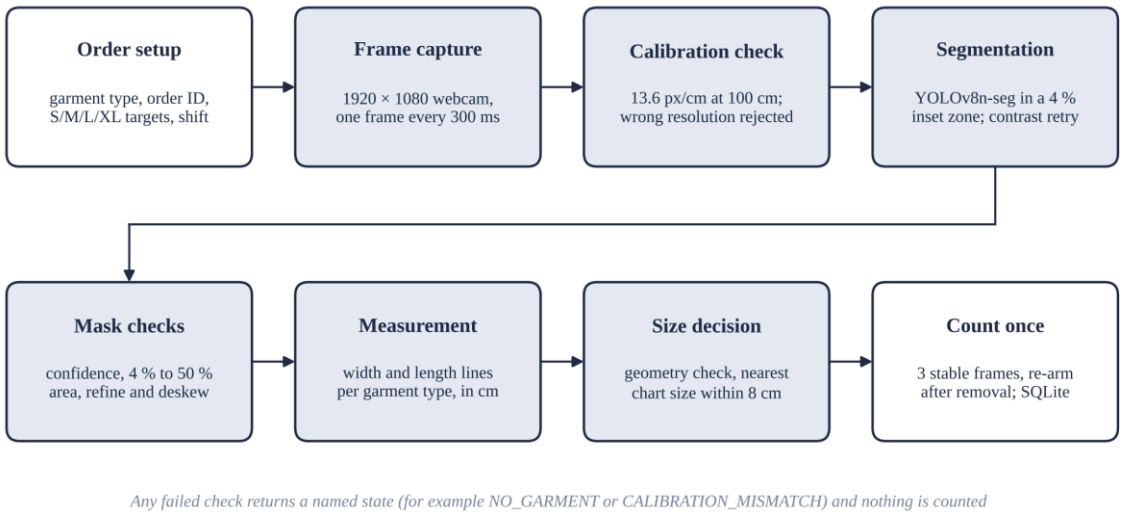


Figure 3.6: Measurement pipeline of Component 4, Garment Size Detection

Each candidate mask must pass a confidence threshold (0.60, or 0.38 for shirts) and cover between 4 % and 50 % of the zone. The accepted mask is refined and rotated upright, and two straight measurements are taken. For T-shirts and shirts the width is a horizontal line across the body just below both sleeves and the length runs from the centre of the collar to the centre of the hem; for trousers the width is taken across the waistband and the length along the longer outside seam, because the centre line ends at the crotch. The measurements are converted to centimetres and checked against the size chart, so that a mask that captured only part of a garment, or an implausible shape, is rejected rather than sized. The size is the chart entry with the smallest Euclidean distance in width and length, weighted 0.7 to 0.3 towards width for shirts because chest width separates adjacent shirt sizes better; if even the nearest entry is more than 8 cm away, the garment is reported as UNKNOWN. The chart covers five T-shirt sizes, seven shirt sizes from XS to 3XL and five trouser sizes; garment size designation in industry follows anthropometric standards such as ISO 8559 [27], and the chart in use is a local reference that a factory would replace with its buyer's specification.

Counting follows the same evidence based principle as the other components. A measurement is counted only when three consecutive frames agree within 1.5 cm in width and 2.0 cm in length, and the next garment is armed only after three

consecutive frames show the garment removed and the scene differs visibly from the counted one, so that a stationary garment is not counted twice after a momentary segmentation failure. Counts, measurements and production orders are stored in SQLite and restored when the service restarts.

Figure 3.7 shows the C4 packing production dashboard during a live session on 11 October 2026. The operator has selected shirts for order mas-333 with a production window of 08:00 to 17:00; the summary cards show the pieces packed in the session against the full target, the size wise table tracks packed and remaining pieces for S, M, L and XL with an at risk status, the production forecast estimates completion from the current packing pace, and the live automatic scanning panel shows the detection zone of the fixed 100 cm camera with the current measurement.

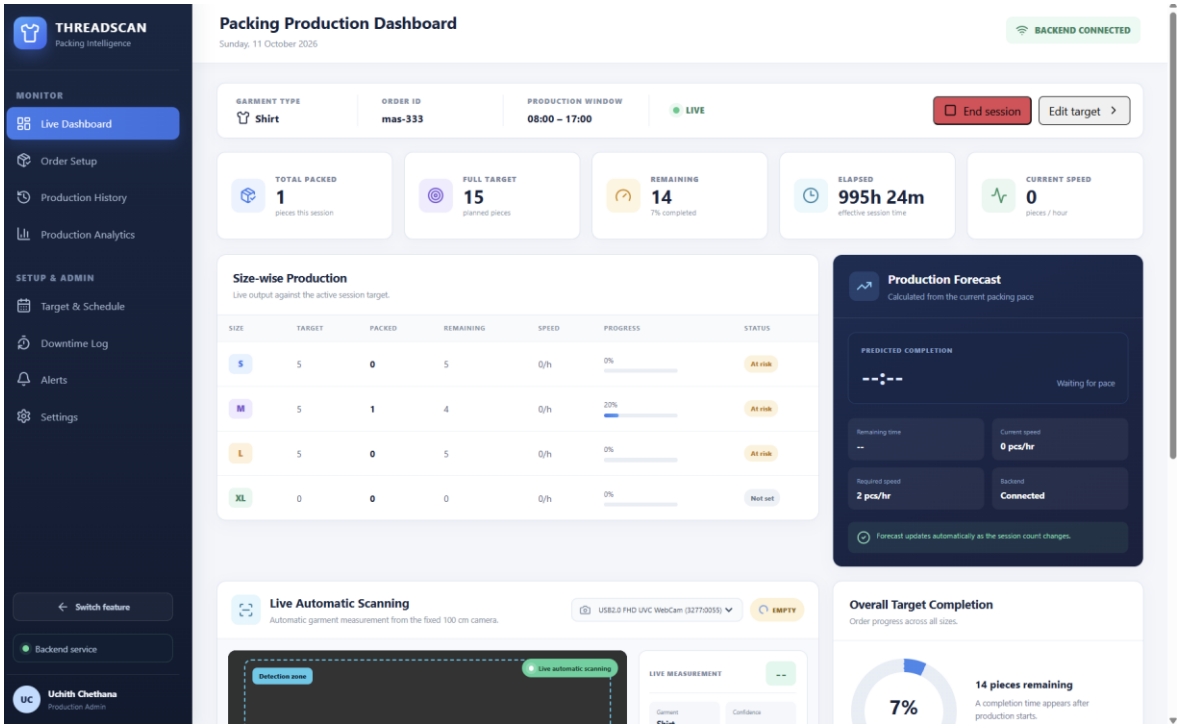


Figure 3.7: Component 4 packing production dashboard during a live session

3.7 End to end user and data flow

Figures 3.8 and 3.9 show the two shared screens of the platform. The landing screen presents Garment Intelligence as one platform for every production stage, summarises the four integrated features and their production areas, and states that processing is local. The workstation selection screen then offers four cards, Workstation 01 Rework Counter for quality checking, Workstation 02 Production

Monitoring for the sewing area, and Workstations 03 and 04 for style and colour detection and size detection in the packing area, each listing its functions, and reminds the operator that the workstations run as isolated services. Figure 3.10 shows how an operator moves through the platform. The same flow applies at every station: the operator chooses the workstation, the platform starts the matching service and shows a startup screen with progress, retry and back options, the workspace opens with its camera and session controls, and leaving the workspace stops the camera and returns to the selection screen.

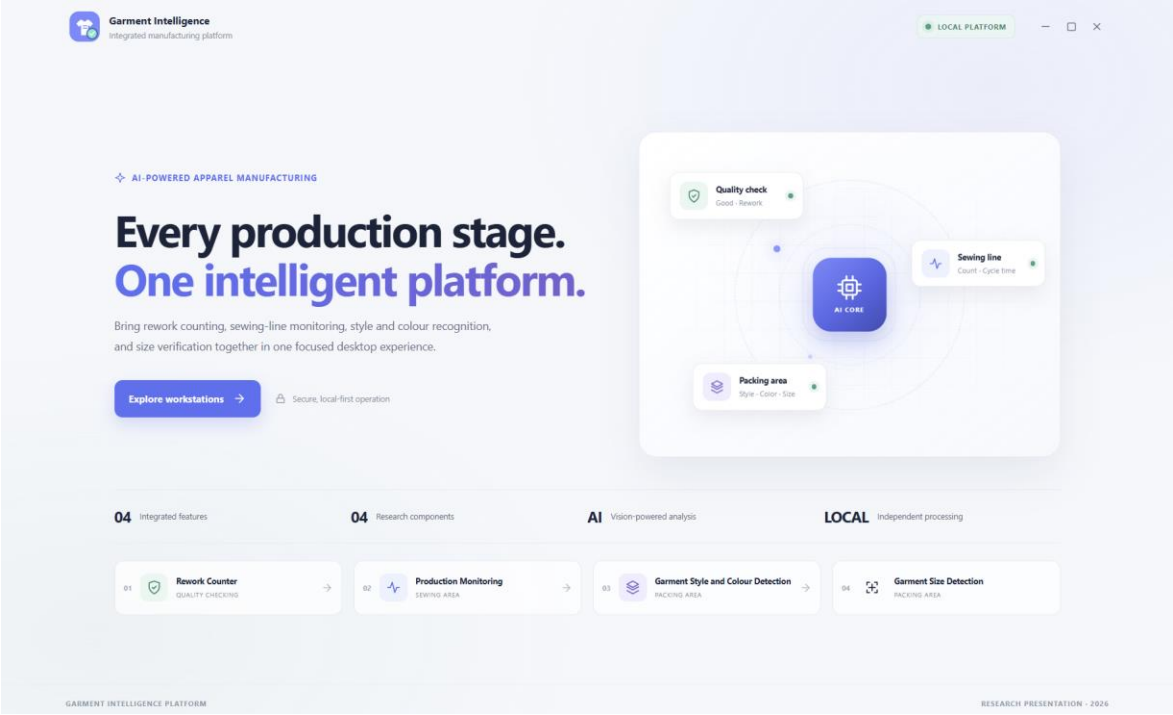


Figure 3.8: Landing screen of the integrated platform

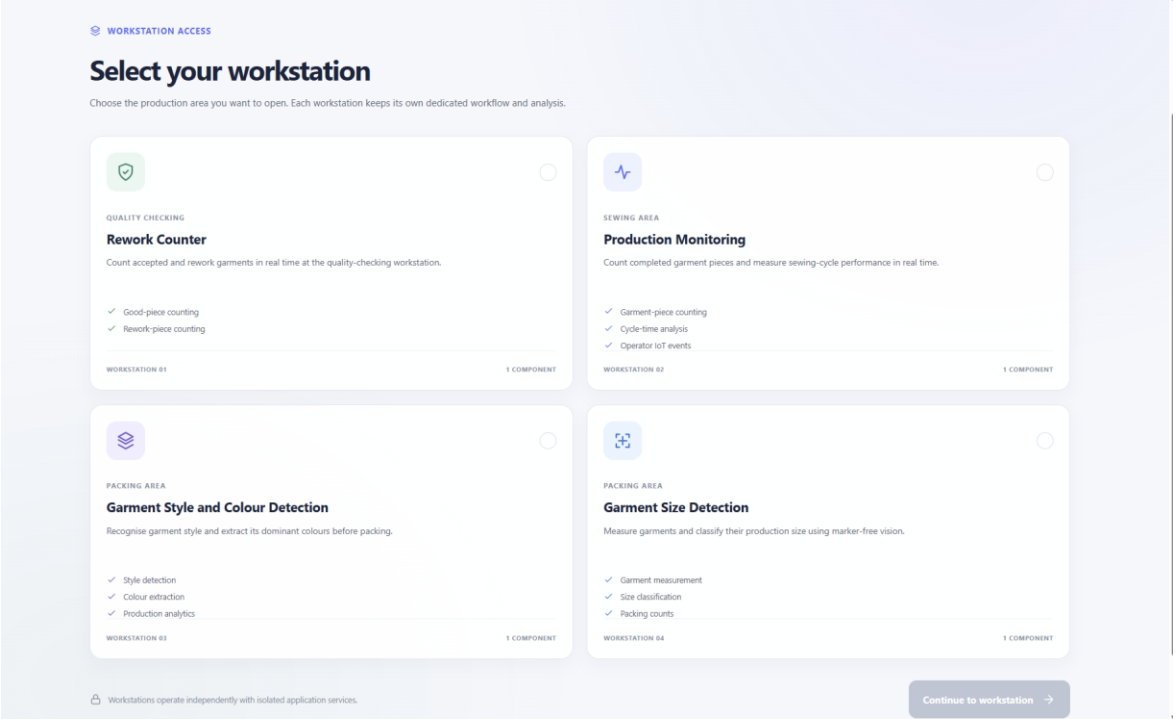


Figure 3.9: Workstation selection screen with one card for each component

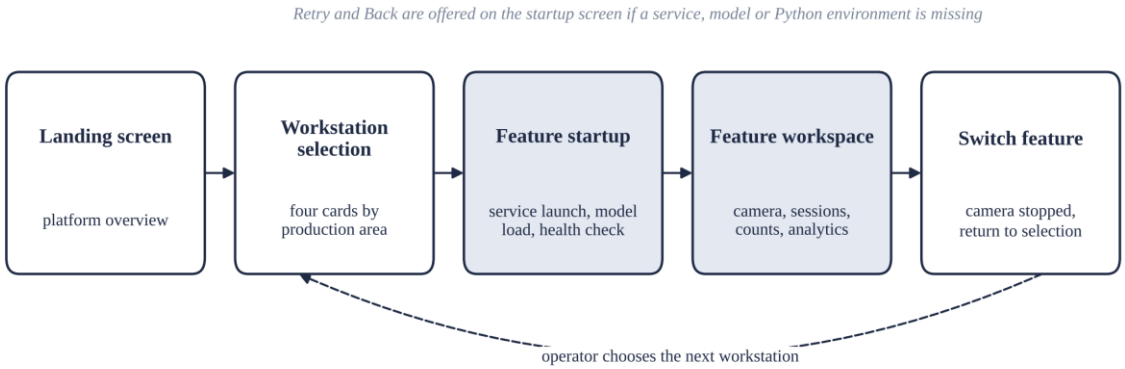


Figure 3.10: Operator flow through the platform

Data flows through the components in parallel rather than in sequence. A garment that is sewn at a C1 workstation, checked at a C2 table and packed past C3 and C4 produces a completion event, a verdict, a garment record and a size measurement in four separate databases, each on its own station computer or in its own data folder. Within a component, the flow is the same in each case: camera frames are reduced to evidence by a model, accepted as a record only when that evidence passes the component's rules, stored with a timestamp, and summarised for the supervisor. The four records are not yet linked to each other by a shared order or garment identifier;

Section 4.2 discusses why, and Chapter 8 identifies this link as the most important integration step for future work.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

The integration followed one rule: preserve each member's ownership of their research artefact. Every component was developed first as a standalone application, with its own interface, service, model and data, and the integrated platform brings these applications into one shell without merging their code. This keeps each individual contribution identifiable and testable, prevents a change in one component from breaking another, and allows each component's evidence to remain valid after integration. The price is that the platform must manage four services with different histories, and the strategy therefore concentrates on the boundary between the shell and each service.

Three integration patterns resulted, shown in Table 4.1. The pattern chosen for each component reflects how much its original service had to change.

Table 4.1: Integration patterns used for the four components

Pattern	Components	How the service is started and reached	Security at the boundary
Authenticated sidecar	C1	Started by BackendManager on a free loopback port with a fresh 32 byte bearer token; every request goes through IPC; video through the garmentstream protocol; Excel export through the main process	Token required on every request; IPC sender check; validation-only rules for simulated controller events
Generic feature sidecar	C2, C4	Started by FeatureBackendManager on a free loopback port; health polled every 350 ms; requests through IPC; C2 video through the garmentrework protocol; C4 frames through a measure channel limited to 15 MB	Per feature allow list of method and path; IPC sender check; loopback only
Preserved original split	C3	Started by StyleColourBackendManager as a FastAPI data service on port 8000 and a Flask vision service on port 5050; the workspace calls both directly over HTTP	Loopback only; renderer content security policy; no token or allow list

Integration proceeded in increments, one component at a time, with the platform shell and C1 as the first pair because C1 required the most complex boundary: authentication, a video stream, Excel export and a Bluetooth controller. C2 and C4 were then added through a generic manager written for services that need only a port, a model path and a data directory. C3 kept its original two service structure because

its vision service and data service were already designed to run side by side, and rewriting them would have changed the system its thesis evaluates. Version 3.1 of the platform recorded the last group of integration fixes: native camera selection in Production Monitoring, the restored launcher for the style and colour vision service, a fully interactive size detection sidebar inside the frameless window, a consistent startup screen for every feature, and validated, SQLite backed size orders.

Five integration risks were managed explicitly. Camera contention was handled by allowing only one camera heavy feature at a time and stopping the outgoing feature's camera before the next one starts. Style collisions between four separately designed interfaces were handled by scoping each feature's styles, for example under a garment-size-module class, and by generating shared utility classes from all feature folders. Dependency conflicts were handled by one shared Python environment with a pinned OpenCV version, so that camera device order and capture behaviour are the same for all features. Installation was handled by bundling each service as its own executable with PyInstaller [28] and recording checksums of the model files. Data safety was handled by keeping every working database in the user data directory, so that an application update cannot overwrite research results.

4.2 Interfaces data contracts and shared models

The interfaces of the platform are deliberately small. The shared contract between the shell and every component consists of four elements, summarised in Table 4.2 and specified fully in Appendix A: a start operation that returns a status, a stop operation that releases the camera, a health probe used to decide when a service is ready, and a set of environment variables through which the shell tells the service where its model and data are and which port to use.

Table 4.2: Shared contract between the platform shell and each component

Element	Definition	Owner
Service status	state is one of starting, ready, stopped or error, with a human readable message shown on the startup screen	Platform shell
Start and stop	IPC calls such as startReworkService and stopReworkCamera; stop is always called before leaving a workspace	Platform shell and component
Health probe	/api/health (C1, with token), /health (C2, C4), / and /api/pipeline/status (C3)	Component
Configuration	Environment variables for port, model path and data directory, for example GARMENT_COUNTER_PORT, GARMENT_REWORK_MODEL_PATH, GARMENT_MODEL_PATH and THREADSCAN_DATA_DIR	Platform shell
Data location	user data directory / features / component, outside the installed application	Platform shell
Error responsibility	The service reports errors as HTTP status and JSON; the shell turns start failures into the error state with the last log line; the workspace shows retry and back	Shared

Inside each component, requests are validated before they reach the domain logic. C1 uses Pydantic models and a bearer token for every route. C2 validates its start and settings bodies with Pydantic, and the shell accepts only five of its paths. C4 checks garment type, upload size (10 MB) and image validity before inference, and validates order payloads field by field, including dates, times, statuses and the S, M, L and XL targets. C3 validates its garment records, settings and downtime events with Pydantic and enforces enumerated values in its database.

The components do not share a domain model, and this was a conscious decision for the present version. Each component's records describe a different event: a completed sewing operation, an inspection verdict, a packed garment, or a measured garment. Forcing them into one schema before each had been validated would have coupled four research projects that were still changing. The consequence is that the platform can show each stage's figures but cannot yet answer cross stage questions, such as how many of the garments sewn by one line for one order were later reworked, packed and sized. Answering them requires a shared order identifier, and ideally a garment identifier, carried through all four stages; this is discussed in Section 6.6 and Chapter 8.

4.3 Shared nonfunctional requirements

Table 4.3 lists the nonfunctional requirements shared by all four components and how the platform addresses them.

Table 4.3: Shared nonfunctional requirements and how they are addressed

Quality	Requirement	How it is addressed
Performance	Interactive frame rates on ordinary computers	Nano scale models (YOLOv8n, YOLOv8n-seg, MobileNetV3-Small); model loaded once per service and warmed up; C3 measured at about 27 ms per frame on a CPU
Usability	No extra work for operators; one way into every feature	Measurement from normal work; one landing and selection flow; shared startup screen with retry and back
Security	No network exposure; least privilege	Loopback only services; context isolated preload; IPC sender checks; C1 bearer token; C2 and C4 allow lists
Privacy	No surveillance of individuals	No video stored by C1, C2 or C4; C3 stores garment crops only; counts per station, session or order, not per person ranking
Reliability	One failure must not stop the platform	Separate processes; health checks; start failures shown with the last log line; camera released on exit
Maintainability	Each owner can change their component	Separate folders, services, models and databases; component tests runnable without hardware
Portability	Windows and macOS presentation machines	Shared Python 3.12 environment; per platform sidecar executables; pinned OpenCV
Transparency	Every figure explainable	Annotated live feeds; confidence and uncertainty states; stored evidence such as images, timestamps and measurements

4.4 Integrated workflow and exception handling

In normal operation an operator starts the application, chooses a workstation, waits for the startup screen to report that the service is ready, sets up the session or order, and works. The service measures, records and summarises; the workspace shows live figures and history. Table 4.4 lists the main failure paths and the platform's behaviour in each.

Table 4.4: Failure paths and recovery behaviour

Situation	Behaviour	Component
Shared Python environment or model file missing	Start refused with a message naming the missing item; startup screen shows retry and back	All
Service does not answer its health probe in time	Error state and process stopped: C1 after 30 s in development or 90 s installed; C2 and C4 after 60 s or 120 s; C3 after 120 s	All
Camera cannot be opened	Start returns an error and no session is created	All
Camera view invalid or uncertain	No event is produced; the state is shown as invalid or uncertain	C1 (workstation gate), C3 (gates), C4 (named states)
Video preview interrupted	Preview reconnects with a fresh stream address	C2; C1 through its protocol
Workflow video ends	Session marked stopped and capture released	C1, C2
Bluetooth controller disconnects	Disconnection event recorded; production readiness withdrawn until reconnection	C1
Frame from a different camera resolution	CALIBRATION_MISMATCH reported; nothing measured	C4
Operator leaves a feature	Camera stopped before the selection screen returns	All
Service restarts	Counts and recent history restored from SQLite	C1, C3, C4; C2 history per session

4.5 Team contribution and dependency matrix

Table 4.5 maps each member to their component, research and development work, integration work and test evidence. Each member owns one component end to end. The platform shell, landing and selection screens, release pipeline and the integration fixes were developed jointly; the shared work column records each member's part in it as shown by the repository history described in Section 5.4.

Table 4.5: Team contribution matrix

Member	Component and research	Development	Integration and shared work	Test evidence
Dharmarathna P.W.A.N. (IT21255892)	C1: annotated 60 videos; state classifier, workstation detector and completion decoder	Counter service, sessions, employees, analytics, Excel export; ESP32-C3 controller integration	Frontend scaffold, login and setup pages (May 2026); first local desktop application (30 August 2026); review and merge of C1 and C4 pull requests	74 backend tests; held out and event evaluation
Pathiraja H.P.M.H.D.S. (IT22627582)	C2: garment piece detector; zone and direction verdict logic	Rework service, live and analysis views, workflow video input	C2 service and workspace brought into the platform through the generic feature manager and the garmentrework video stream	51 backend tests; 3 frontend tests; replay and simulation
Jayaweera M.P.Y.C. (IT22556370)	C3: four class detector; capture gates; colour method; analytics	Perception and data services; dashboard; downtime and order analytics	Repository owner; main process and shared index updates; final integration merges, pull requests #19 to #29 (September to October 2026)	107 tests in original repository; bench trial
Jayarathna W.G.U.C.G. (IT22187796)	C4: garment segmentation model; calibrated measurement; size classification	Size service, order management, analytics and history views	Layout fixes and regional size chart (May 2026); size feature integration, pull request #17 (31 August 2026); final updates (9 October 2026)	3 backend tests; 3 frontend order tests

The dependency matrix in Table 4.6 shows that the components depend on the shared platform but not on each other. This is the property that allowed four members to work in parallel and to integrate one component at a time.

Table 4.6: Dependency matrix (a mark means the row depends on the column)

	Platform shell	Python env.	C1	C2	C3	C4	ESP32-C3
C1 Production Monitoring	✓	✓					✓
C2 Rework Counter	✓	✓					
C3 Style and Colour	✓	✓					
C4 Size Detection	✓	✓					
Platform shell		✓					

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

Table 5.1 lists the technologies shared by the platform and the reasons for the main choices.

Table 5.1: Common technology stack

Layer	Technology	Reason for selection
Desktop shell	Electron 43, Electron Forge, webpack, TypeScript 5.9	One installable application with native windows, process supervision, custom protocols and Bluetooth access
User interface	React 19, Tailwind CSS 4, Recharts and Chart.js, Lucide icons	Component model shared by four separately designed interfaces
Services	Python 3.12, FastAPI with Uvicorn and Pydantic [29]; Flask for the C3 vision service	Validated REST endpoints, streaming responses, small footprint
Vision and learning	PyTorch, torchvision, Ultralytics YOLOv8 [5], OpenCV 4.11.0.86 (pinned), Supervision	Real-time detection, segmentation and tracking with fine tuning support
Persistence	SQLite, one database per component [30]	Zero configuration, transactional, one file per component
Hardware	USB webcams (for example Logitech C922 for C4); ESP32-C3 with Bluetooth Low Energy [23], [31]	Low cost, widely available, no tagging
Packaging	PyInstaller sidecar per component; Squirrel and ZIP makers	Operators need neither Python nor Node.js
Testing	pytest with FastAPI TestClient; Node test runner with TypeScript stripping	Service, logic and integration tests without hardware
Collaboration	Git and GitHub; Microsoft Teams and Planner	Version control, task tracking and meeting records

The repository mirrors the ownership structure. Under frontend, the shared shell lives in src, and each component has its own folder: garment-counter-frontend, garment-rework-frontend, garment-style-colour-frontend and garment-size-frontend. Under backend, each component has its own service folder, and one script creates the shared Python environment with all four services installed in editable mode. Under models, each component's trained weights are kept in their own folder and are loaded only by that component's service.

5.2 Development and integration process

The project followed an iterative, design science approach [32]: each member built, evaluated and revised their component, and the group integrated the components in

increments. Work was planned in Microsoft Teams and Planner, and each member kept a weekly log book. Integration was interface first: before a component was moved into the platform, the group agreed the start, stop, health and configuration contract for it, the IPC calls it would need, and where its data would live. Each component remained runnable as a standalone service throughout, which allowed its owner to continue research while integration proceeded.

Quality was protected by automated checks at two levels. Each component carries its own service tests, and the platform carries integration tests that guard the rules of the integrated product: that every component has its own frontend, backend and model folder; that all four backends are installed into the shared environment; that the trained models stay distinct; that styles remain scoped; that the video streams are allowed by the content security policy and are not cut off by a timeout; and that specific regressions found during integration, such as a camera mapping error in C1 and a non interactive sidebar in C4, cannot return. A single verify command runs type checking, linting, the tests and a build check before a demonstration.

5.3 Deployment and operational configuration

For development, a member creates the shared Python environment with one script, installs the frontend packages and starts the application with npm start. For release, the pipeline builds an isolated executable for each of the four services on the target operating system and processor architecture, validates that all required model and branding files are present, records checksums of the models in a release manifest, compiles the renderer and creates the platform installer. Installers must be built on the same operating system and architecture as the presentation machine, and the manifest check refuses to run a service built for a different platform.

At run time all four services listen only on the loopback interface. C1, C2 and C4 use ports chosen at start up, while C3 uses ports 8000 and 5050. In an installed build each service writes its database to its own folder under the user data directory, so an update or reinstall does not touch production data, while the seed databases remain in the backend folders of the source. Backups are a matter of copying these folders, and restoring them restores each component's history. The services write their logs to the

main process, which keeps the last lines of each to explain a failed start. Monitoring beyond this, such as a central health dashboard for several stations, is outside the present scope.

5.4 Evidence of team execution

The execution of the project is evidenced by the shared repository, whose structure, tests and version history record each member's work and the integration steps; by the four individual theses; by the research manuscript prepared for C1 [24]; and by the trained models and their training records. Table 5.2 summarises the models.

Table 5.2: Trained models used by the integrated platform

Component	Model and task	Training record	Checkpoint
C1	YOLOv8n workstation detector (1 class); MobileNetV3-Small temporal classifier (2 states)	Detector 50 epochs at 640 px; classifier early stopped at epoch 12, best epoch 5 [10]	best.pt; best_model.pt
C2	YOLOv8n garment piece detector (1 class)	50 epochs, batch 8, 640 px; trained 15 August 2026 with Ultralytics 8.4.28	best.pt (6.25 MB)
C3	YOLOv8n style detector (4 classes)	50 epochs on a CPU; best epoch 41 [12]	best.pt (about 6.25 MB)
C4	YOLOv8n-seg garment segmentation (1 class)	50 epochs, batch 2, 640 px, seed 42; Roboflow dataset version 5, 1,295 images (1,070 training, 141 validation, 84 test) [13]	best_model.pt (6.76 MB)

Work was planned in Microsoft Teams under the R26-IT-054 team, with one Planner plan for each feature. Figure 5.1 shows the plan for C3 as an example. Each plan breaks the component into dated tasks grouped into buckets: the Packing Zone bucket runs from defining the video data requirements and obtaining factory access and recording permission (25 March to 8 April 2026), through recording, cataloguing, frame extraction, annotation schema and annotation (April and May), to merging the annotations and reporting the class distribution (May); the Best Frame Capture bucket then covers the capture logic, the layered capture gate, region of interest cropping, blur rejection by the variance of the Laplacian and box area plausibility checks (5 to 20 June 2026). All the tasks shown are marked complete.

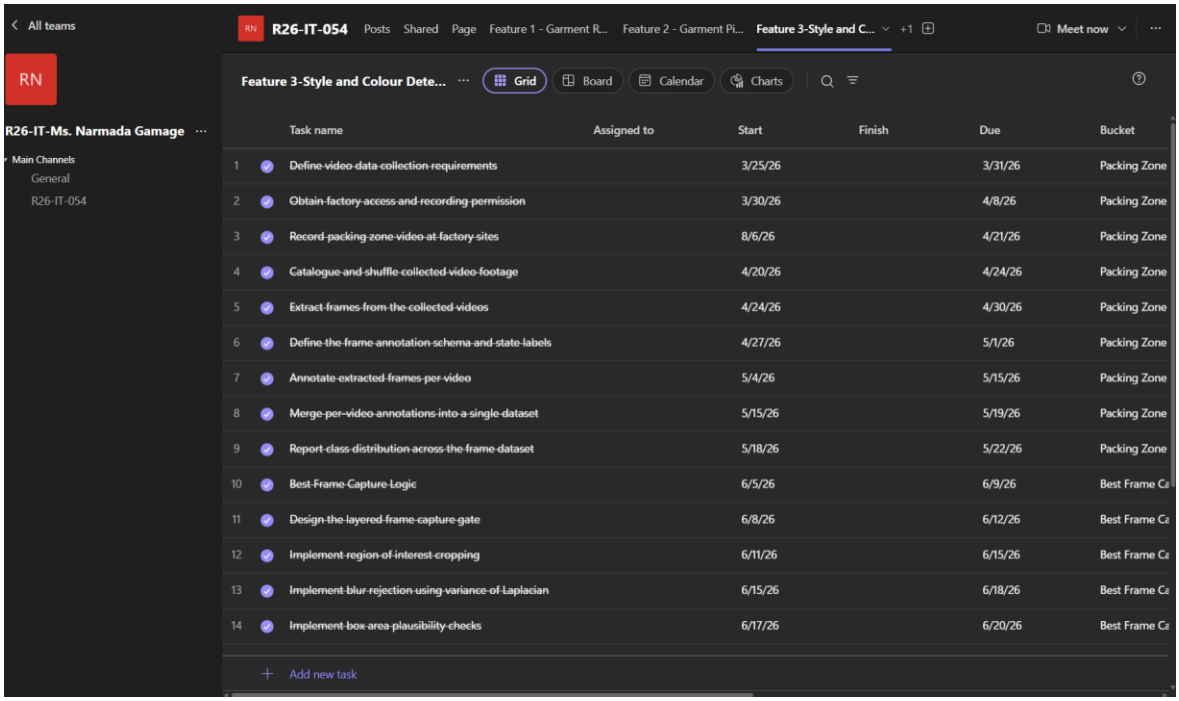


Figure 5.1: Microsoft Planner plan for Component 3 in the R26-IT-054 Teams workspace

The shared GitHub repository, yasasArt/R-26-IT-054, records the integration. Its first commits on 4 May 2026 set up the shared React frontend with its configuration, types, stores, login and setup pages, and twelve pull requests merged in May brought the first sewing, quality checking, colour and size screens together. Model work followed: the 60 C1 videos were annotated and split on 21 July and the MobileNetV3 classifier was trained on 23 July; the C3 model, training folder and webcam pipeline were committed between 9 and 18 August; and the C4 video annotation, best frame selection models, size chart and size detection code between 17 and 21 August. The last two weeks of August were the main integration period, when weekly commits peaked at about 36 (Figures 5.2 and 5.3). The C1 FastAPI foundation, SQLite schema and YOLOv8 workstation detector were committed on 26 August, the ESP32-C3 controller code and production session lifecycle on 27 August, the temporal cycle decoder and live vision runtime on 29 August, and the local desktop application on 30 August; the size feature was merged as pull request #17 on 31 August. In September the C1 operation type model was trained and compared with the baseline (16 to 25 September) and C3's database was migrated to SQLite, and the

final analytics and integration updates were merged on 9 and 10 October 2026, ending with pull request #29.

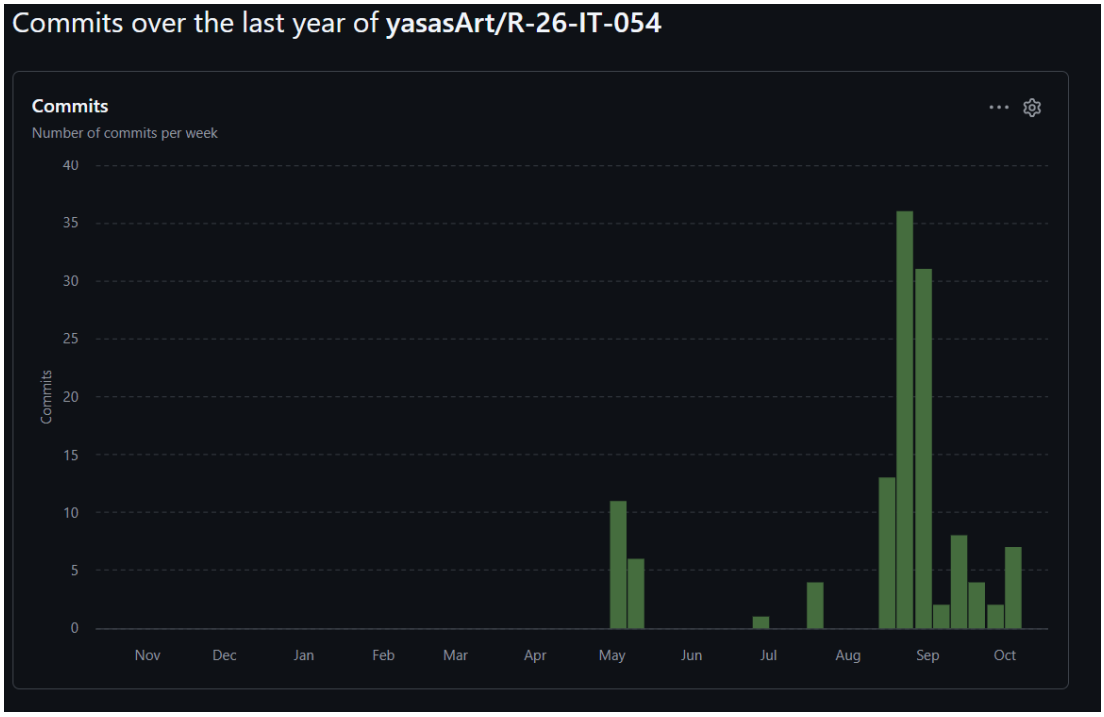


Figure 5.2: Commits per week to the shared repository over the project year

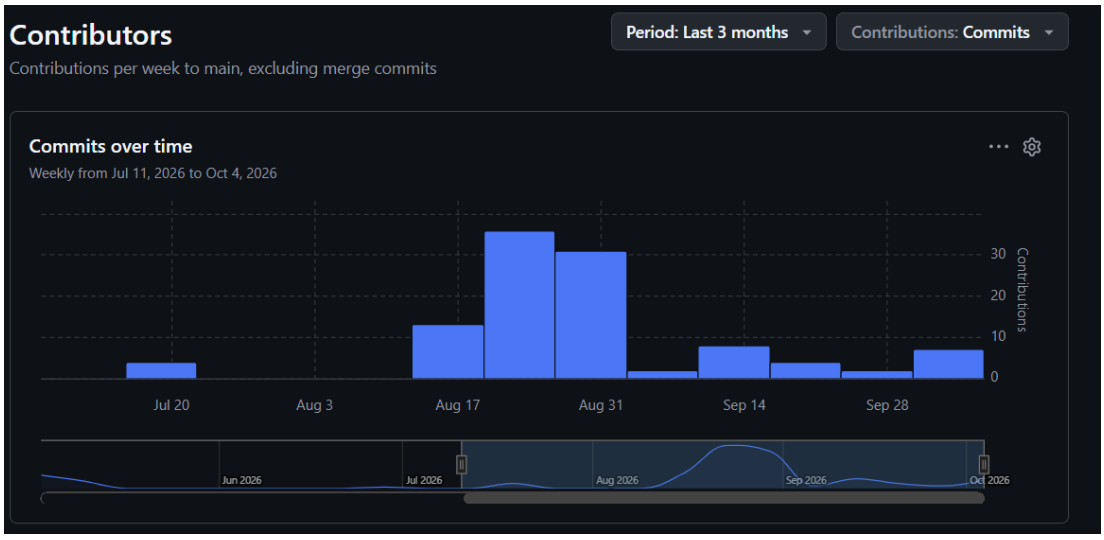


Figure 5.3: Weekly commits to the main branch in the last three months, excluding merge commits

Figure 5.4 shows the contributions to the main branch over the last three months, excluding merge commits: 41 commits by UchithChethana (Jayarathna W.G.U.C.G.), 37 by yasasArt (Jayaweera M.P.Y.C.) and 29 by asitha-dharmarathne (Dharmarathna P.W.A.N.). The repository was created after the components had

begun, so earlier development is documented in the individual theses rather than in these figures. No commits appear under the name of Pathiraja H.P.M.H.D.S.; the C2 service and workspace were added to the integrated platform as their own component folders, and their development and evaluation are documented in the C2 thesis [11]. Appendix D reproduces the commit graph of the repository.



Figure 5.4: Commits by contributor to the main branch in the last three months

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The evaluation asks five group level questions, each tied to the group objectives in Table 2.2. Because the four components measure different events, there is no single accuracy figure for the platform; instead, each question is answered with the kind of evidence that can actually support it. Table 6.1 sets out the questions, success criteria and evidence. Throughout, four levels of evidence are kept apart: model evidence (held out or validation metrics), logic and software evidence (automated tests), integration evidence (tests and runs of the combined application), and field evidence (independent counts at a working line). A claim is made only at the level its evidence supports.

Table 6.1: Group evaluation questions, success criteria and evidence

Question	Success criterion	Evidence used	Objective
EQ1 Do the models recognise what each station needs?	Held out or validation metrics high enough to drive counting (mAP@0.5 or accuracy above 0.90)	Individual theses; checkpoint training records	GO1, GO2
EQ2 Does each component turn model output into correct records exactly once?	Logic tests pass, including duplicate, missing and invalid input cases	Component test suites; replays; simulation	GO1, GO2
EQ3 Are records turned into correct decision information?	Analytics match hand calculations and documented definitions	Analytics tests; worked examples	GO3
EQ4 Do the components operate as one secure platform?	All integration tests pass; services isolated and loopback only	Platform test suite; integrated runs	GO4, GO5
EQ5 Do the counts match reality at a working line?	Independent count agreement over a multi week trial	Not yet available	GO1, GO6

6.2 Integration and end to end testing

For this report the automated test suites of the integrated repository were executed on 10 October 2026 on a Linux workstation with Python and Node.js, without cameras or a graphics processor. Figure 6.1 and Table 6.2 summarise the results. All 188 executed tests passed. The platform suite of 60 tests in 12 files covers the integration rules of the product; the C1 suite of 74 tests exercises the authenticated service, including tests that load the two real trained checkpoints and run a real clip through the classifier; the C2 suite of 51 tests, prepared with the C2 thesis and to be added to

the integrated repository, exercises the real rework service with only the camera, model and tracker replaced by stubs; and the C4 suite tests its order database. The C3 suite of 107 tests is part of the component's original repository and was reported passing in its thesis [12]; it is not yet included in the integrated repository.

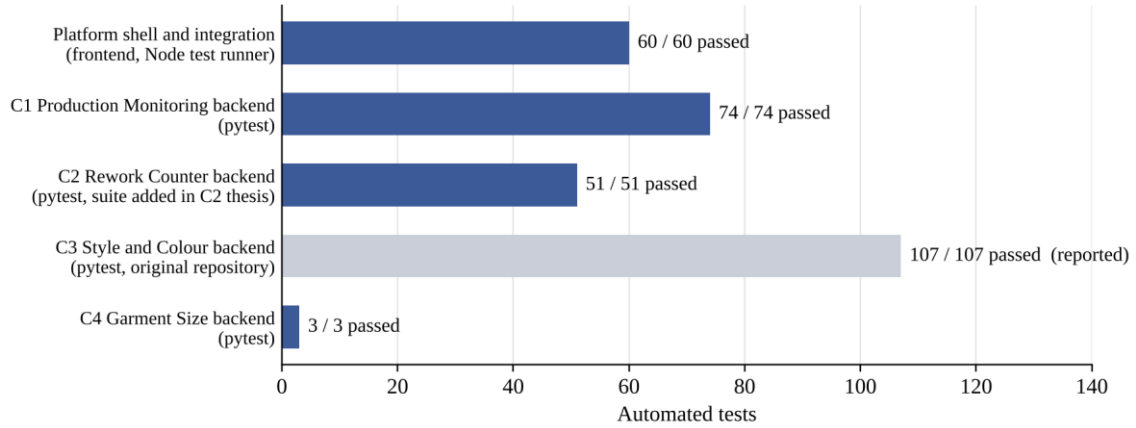


Figure 6.1: Automated test results by suite

Table 6.2: Automated test suites of the integrated solution

Suite	Tests	Result	Main coverage
Platform shell and integration (Node)	60	60 passed	Feature registry and ownership; shared environment; model isolation; scoped styles; startup and connection policy; release manifests and checksums; Bluetooth policy; production readiness rules; rework stream and video picker; size order persistence
C1 Production Monitoring (pytest)	74	74 passed	Authentication; sessions and employees; controller events, disconnection and reset; camera mapping; real checkpoint loading; vision pipeline; decoder rules; analytics and Excel export
C2 Rework Counter (pytest)	51	51 passed	HTTP interface; zone and verdict logic; persistence; documented limits
C4 Garment Size Detection (pytest)	3	3 passed	Order persistence, fresh sessions and deletion
Executed for this report	188	188 passed	
C3 Style and Colour (pytest, original repository)	107	Reported passed [12]	Analytics, database, vision functions and integration

Three results deserve emphasis because they concern the interaction between components and the platform rather than a single component. First, the platform suite enforces the isolation rules that make parallel ownership possible: it fails if a component loses its own frontend, backend or model folder, if two components share trained weights, or if a component's styles leak into another. Second, several tests guard regressions that only appeared after integration, such as the C2 video proxy

that aborted the stream after ten seconds, the C1 camera mapping that confused browser and native camera order, and the C4 sidebar that could not be clicked inside the frameless window. Third, the C1 suite now contains 13 decoder tests, covering short predictions, low confidence uncertainty, cooldown, recovery after an invalid view and the cancellation of an unfinished cycle; these are the behaviours that the C1 thesis listed as remaining verification work [10], so integration has closed part of that gap at the software level. The integrated C1 service also measures the first cycle of a session from the first observed sewing start, where the thesis evaluation left the first completion without a cycle; this difference in definition should be kept in mind when the two are compared.

End to end operation was checked by running each component inside the integrated application with its camera or a workflow video: C2's stored sessions show the complete chain from video through the service to the session history [11], and C3's bench trial ran camera, perception, data service and dashboard together [12]. The integrated application was also run with cameras attached during the final checks in October 2026, on Windows and macOS presentation machines, and Figures 3.2 to 3.9 show screens captured in these runs: the C1 workstation preparation screen with a passed camera test and five of six readiness checks, the controller being the one not yet connected; the C2 live production workspace with its input source and zone settings; the C3 camera preparation screen and live decision support dashboard; the C4 packing dashboard counting a shirt against order mas-333 on 11 October 2026; and the shared landing and workstation selection screens. These runs confirm that each card starts its service, loads its model and opens its workspace inside the one shell. They are demonstrations rather than measured trials, and the counts visible in them are not used as accuracy evidence.

6.3 Overall results

Table 6.3 brings together the headline results of the four components. They are not directly comparable, because each measures a different task on a different dataset, but together they show that every station's model reached the level needed to drive its counting logic on held out or validation data.

Table 6.3: Headline results of the four components

Component	Model result	Counting or measurement result	Source
C1 Production Monitoring	State classifier: 98.10 % accuracy, 97.76 % macro F1 on 1,001 held out clips; workstation detector: 99.50 % mAP@0.5, 75.80 % mAP@0.5:0.95 (validation)	Raw transition baseline on 9 test videos: 59 of 66 completions matched, precision 92.19 %, recall 89.39 %, F1 90.77 %, count MAE 0.67 pieces	[10]
C2 Rework Counter	Detector: mAP@0.5 0.938, precision 0.895, recall 0.869 (validation)	600 of 600 simulated garments correct on clean tracks; identical verdicts in two replays of one workflow video	[11]
C3 Style and Colour	Detector: precision 0.962, recall 0.955, mAP@0.5 0.958 (held out test); about 27 ms per frame on a CPU	Bench trial: 37 captures, all at confidence 0.85 or higher; worked example analytics matched hand calculation	[12]
C4 Size Detection	Segmentation: mask precision 0.977, recall 0.970, mAP@0.5 0.960, mAP@0.5:0.95 0.955 (validation, final epoch of 50)	Five tape comparisons of T-shirts and shirts: width MAE 0.570 cm, length MAE 0.674 cm, largest error 1.47 cm	[13]

For C4, Figure 6.2 shows the absolute width and length errors of the five tape comparisons reported in its thesis [13]. The segmentation model was fine tuned from YOLOv8n-seg weights for 50 epochs on the 1,295 image dataset; at the final epoch the box and mask mAP@0.5:0.95 were 0.952 and 0.955, and this near equality indicates that the masks fit the garments tightly, which a measurement based on the mask outline needs. Against the tape, the width MAE was 0.570 cm (RMSE 0.759 cm) and the length MAE 0.674 cm (RMSE 0.847 cm), and no error exceeded 1.47 cm. These figures are preliminary: records 1 and 2 are repeated trials of one garment, trousers were not included, reference sizes were not recorded, and the model has not yet been evaluated on its 84 test images. They show that the calibrated measurement chain works on real garments rather than establishing its accuracy across garment types [13].

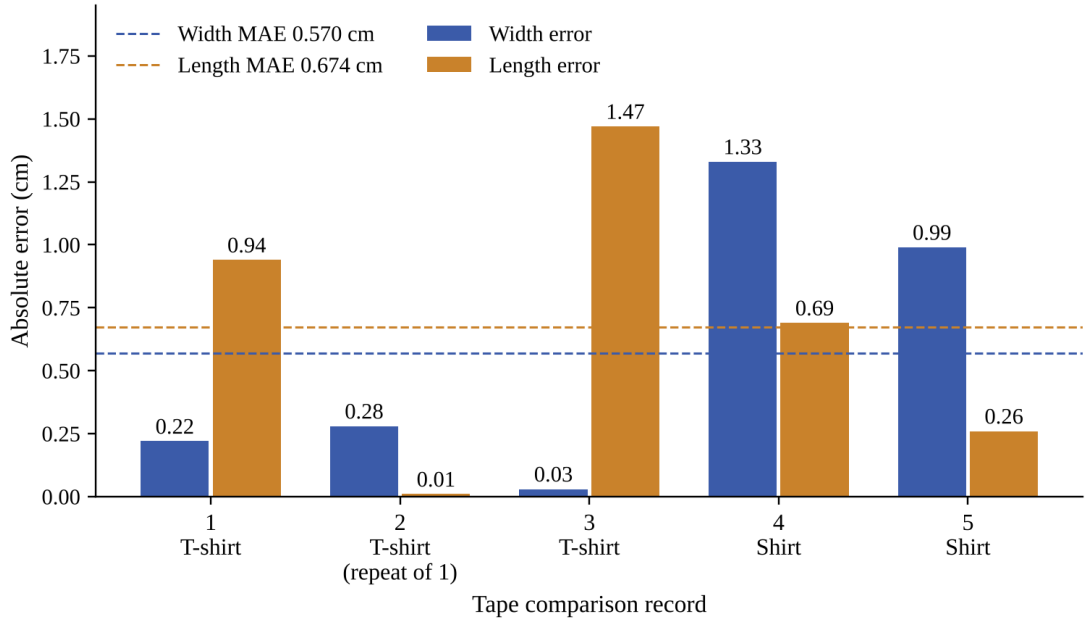


Figure 6.2: Absolute width and length errors of Component 4 in five tape comparisons

6.4 Stakeholder and user validation

Stakeholder involvement differed between components. C1's recordings were made with the permission of sewing operators and workplace representatives, and C3's design came from observing manual counting at a factory packing area [10], [12]. These activities shaped the problem and the data, but they are not usability evaluations. No structured study with operators, inspectors or supervisors evaluated the complete platform within the project period, and no claim is made that users prefer it or that it saved measured time. The C4 thesis likewise records that no interview, questionnaire or usability task results were collected for the size component [13].

A stakeholder study of the integrated platform should begin with the meaning of each event at each station, as the C1 thesis recommends: what counts as a completed operation, a rework decision, a packed garment and a correct size. It should then show supervisors examples of correct, missed and false events from each component, because the consequences of each error type differ, and ask whether the figures on each workspace support the decisions they actually make during a shift.

6.5 Achievement of common objectives

Table 6.4: Achievement of the group objectives

Objective	Status	Justification
GO1 Automatic measurement at three stages	Partly achieved	All four components measure their events in real time from normal work; field counting accuracy against independent counts has not been measured
GO2 Quality and attribute information	Achieved	Cycle time (C1), verdict (C2), style and colour (C3) and size (C4) are attached to records and verified by model metrics and tests
GO3 Decision information	Achieved within each stage	Pass rate and delay, efficiency, projection and overtime, order progress by size, and cycle analytics are implemented and tested; cross stage figures need a shared order identifier
GO4 One secure platform	Achieved with qualification	Four isolated services in one shell, 60 integration tests passing; C3 still uses fixed ports and direct HTTP without an allow list
GO5 Ordinary hardware	Achieved	USB webcams, existing computers and a low cost ESP32-C3 controller; nano scale models
GO6 Verified with honest limitations	Achieved	188 tests executed and passing, 107 reported; limitations documented in each thesis and in Section 6.6

6.6 Integrated limitations and threats

- **No field validation of the whole line.** Each component's accuracy has been measured on held out, validation or simulated data; none has been compared with independent counts over weeks at a working line, so the platform's field error rates are unknown.
- **No cross stage linkage.** Records are not joined by order or garment identifiers, so the platform cannot trace a garment from sewing to packing or compute line level yield.
- **Uneven integration patterns.** C3's fixed ports can collide with other software, its manager treats any process already answering on those ports as ready, and its interface is not behind the shell's allow list; C1 alone uses a token.
- **One camera heavy feature at a time.** A single computer cannot run several stations simultaneously; a real deployment needs one computer per station.
- **Dataset and generalisation threats.** Each model was trained on data from limited sites, lighting and garment ranges; C1's split is by video rather than by operator, C4's split independence by source video or garment has not been verified, and C4's scale is valid only at the calibrated height and resolution.

- **Component specific limitations.** These are reported in each thesis, for example C2's dependence of counting on its video stream and C3's reversed forecast weights, and are not repeated here.
- **Test coverage of hardware paths.** Camera, Bluetooth and GPU paths were exercised in demonstrations rather than in automated tests, because the test machine had no such devices.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

Garment Intelligence turns ordinary webcams and the computers already found beside a production line into a live record of production and quality across the line. A supervisor sees sewing output and cycle time at a workstation, the pass rate at the checking table, and packed progress by style, colour and size against the order, during the shift rather than the next morning. Table 7.1 summarises the value for each beneficiary.

Table 7.1: Value of the integrated platform for each beneficiary

Beneficiary	Value
Operators and inspectors	No tally sheets or buttons for routine work; attention stays on the garment; simple controller for rework and downtime
Line supervisors	One place to see counts, cycle time, pass rate, efficiency, delay and overtime advice while there is still time to act
Planners and merchandisers	Progress by style, colour and size against targets; exports for planning
Quality managers	Timed record of rework decisions and pass rate trends
Factory owners	Affordable digitalisation without tagging or automated inspection lines; data kept on site
Society	Less repeated work and waste; fairer measurement that separates downtime from performance

7.2 Market alternatives and differentiation

The target segment is small and medium apparel manufacturers in Sri Lanka and the region [1], which have many sewing, checking and packing stations but cannot justify enterprise manufacturing execution systems or automated inspection. Their present alternatives are paper tallies and spreadsheets, which are cheap but late; tablet or handheld entry, which is timely but adds a step to every garment; barcode or RFID tracking, which gives traceability at the cost of tags, readers and scanning; machine sensors such as current monitors on sewing machines [17], which measure machine activity but not completed work; and automated defect inspection [15], which is powerful but narrow and costly. The platform's differentiation is the combination of four properties: it measures from normal work with no added steps, it

covers several stages with one product, it runs offline on existing hardware, and every figure can be traced to its evidence.

7.3 Business model and feasibility

A suitable model is a per station subscription with a one time installation fee. Installation covers mounting and calibrating cameras, configuring lines and zones, pairing controllers and training supervisors; the subscription covers updates, support and retraining of models for new garment types or sites. Because the software runs on the customer's computers, each additional station costs little to serve, and features can be licensed separately, for example the Rework Counter alone at the checking tables. Partners could include factory automation integrators and industry associations, which can reach many small plants.

Technical feasibility is supported by the components' results on ordinary hardware and by the release pipeline that already produces installers without a Python or Node.js prerequisite. Operational feasibility depends on stable camera mounting and lighting, which installation must ensure. Economic feasibility cannot be established without prices and a pilot; a pilot on one line, run alongside the existing manual records for several weeks, would measure accuracy, time saved and supervisor use, and provide the case study needed for wider sales.

7.4 Adoption intellectual property and compliance

The main adoption barriers are trust in automatic counts, worker concern about cameras, and the effort of installation. The first is addressed by visible evidence and uncertainty in every component and by running the system alongside manual records until agreement is shown. The second is addressed by design: cameras look at work surfaces, video is not stored except for C3's garment crops, records are per station, session or order rather than per person ranking, inspectors' and operators' decisions remain theirs, and downtime is separated from performance. A deployment would also need worker consultation, signage, and compliance with Sri Lanka's Personal Data Protection Act [33] and with buyers' social compliance audits. Where C1 records employee names with sessions, access to that data must be restricted and its purpose stated.

Ownership of each component rests with its author and the institution under the module's rules. The Ultralytics framework used by all four components is licensed under the AGPL 3.0, with an enterprise licence available for commercial products [5]; a commercial release must either comply with the AGPL or obtain that licence, and the licences of the other dependencies must be reviewed in the same way. The platform is a monitoring tool and must remain separate from machine safety control.

7.5 Sustainable Development Goal contribution

The plausible contributions to SDGs 8, 9 and 12 were outlined in Table 1.2. Their realisation depends on how the platform is used. Earlier visibility of rework and delays reduces repeated work and wasted material only if supervisors act on it; fairer measurement supports decent work only if the data is not used to pressure individuals; and affordable technology helps small industry only if installation and support remain affordable. Possible negative effects are increased monitoring pressure on workers and dependence on a single vendor. Measures to sustain the benefits include per station rather than per person reporting by default, worker consultation before deployment, local data ownership, and open export formats so that factories keep control of their records.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

The common problem was that production and quality information at the sewing, checking and packing stages of an apparel line is recorded late, in aggregate and without evidence. The group addressed it with four components, each of which measures a different station's events from normal work with a camera and a lightweight model, and an integrated platform that delivers them as one offline desktop application. On held out, validation or simulated data, every component reached the level needed to drive its counting logic, and the 188 automated tests executed for this report all passed, together with 107 tests reported for C3. The integrated evidence therefore supports the conclusion that timely, explainable production records can be obtained across several stages of a line with webcams, existing computers and small models, without adding work for operators. It does not yet show how accurate those records are at a working line over weeks; that is the main question that remains.

8.2 Principal integrated contribution

Beyond the four components, the group produced three things of value. First, a platform architecture that lets independently owned research components share one product while keeping their code, models, data and evidence separate, with explicit start, stop, health and configuration contracts and an integration test suite that enforces the isolation. Second, a set of shared design principles, applied in each component: measure from normal work, accept a record only on evidence, keep human decisions as the source of truth where judgement is needed, show uncertainty, and keep data local. Third, an honest map of what has and has not been validated, which tells a factory or a future team exactly which experiments to run next.

8.3 Future directions

- Run a multi week pilot at one line with all four stations, comparing each component's records with independent counts, measurements and timings.

- Introduce a shared order identifier, and later a garment identifier, so that sewing, checking and packing records can be joined into line level yield, throughput and delivery forecasts.
- Bring C3 onto the same integration pattern as the other components, with ports chosen at start up and an allow listed or token protected interface, and add its test suite to the integrated repository.
- Allow several stations to report to a central database for a supervisor's line view, while keeping each station able to work offline.
- Evaluate C4 on its 84 held out test images and on a larger set of tape measured garments, including trousers and recorded reference sizes, across colours and fabrics, and replace the local size chart with buyers' specifications.
- Conduct structured usability studies with operators, inspectors and supervisors, and adapt each workspace to the decisions they make during a shift.

Each of these steps builds on the existing contracts and tests, so they can be undertaken without disturbing the components that already work.

REFERENCES

[1] Sri Lanka Export Development Board, "Apparel and textile industry in Sri Lanka," [Online]. Available: https://www.srilankabusiness.com/apparel/

[2] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & Information Systems Engineering*, vol. 6, no. 4, pp. 239–242, 2014.

[3] C. N. Keist, "Quality control and quality assurance in the apparel industry," in *Garment Manufacturing Technology*, R. Nayak and R. Padhye, Eds. Cambridge, U.K.: Woodhead Publishing, 2015, pp. 447–466.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[7] J. M. Juran and A. B. Godfrey, *Juran's Quality Handbook*, 5th ed. New York, NY, USA: McGraw-Hill, 1999.

[8] S. Nakajima, *Introduction to TPM: Total Productive Maintenance*. Cambridge, MA, USA: Productivity Press, 1988.

[9] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

[10] P. W. A. N. Dharmarathna, "Vision based garment piece counting and sewing cycle time measurement," B.Sc. (Hons) individual thesis, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka, 2026.

[11] H. P. M. H. D. S. Pathiraja, "Real time good and rework garment counting at the quality checking workstation using detection, multi object tracking and zone based verdict logic," B.Sc. (Hons) individual thesis, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka, 2026.

[12] M. P. Y. C. Jayaweera, "Real time garment style and colour detection with confidence gated capture and production decision support," B.Sc. (Hons) individual thesis, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka, 2026.

[13] W. G. U. C. G. Jayarathna, "Real-time best-frame selection and marker-free garment size measurement for smart apparel packing," B.Sc. (Hons) individual thesis, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka, 2026.

[14] J. E. See, "Visual inspection: A review of the literature," Sandia National Laboratories, Albuquerque, NM, USA, Rep. SAND2012-8590, 2012.

[15] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2008.

[16] D. C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed. Hoboken, NJ, USA: Wiley, 2013.

[17] W.-K. Jung, H. Kim, Y.-C. Park, J.-W. Lee, and S.-H. Ahn, "Smart sewing work measurement system using IoT-based power monitoring device and approximation algorithm," *Int. J. Production Research*, vol. 58, no. 20, pp. 6202–6216, 2020.

[18] A. Fathy et al., "VARSew: A visual action recognition dataset on garment sewing," Mendeley Data, version 1, Oct. 2023, doi: 10.17632/37b9hg5mg7.1.

[19] Y. Ge, R. Zhang, X. Wang, X. Tang, and P. Luo, "DeepFashion2: A versatile benchmark for detection, pose estimation, segmentation and re-identification of clothing images," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5337–5345.

[20] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2961–2969.

[21] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.

[22] Electron, "Context isolation," Electron Documentation. [Online]. Available: https://www.electronjs.org/docs/latest/tutorial/context-isolation

[23] Espressif Systems, "ESP32-C3 series datasheet," [Online]. Available: https://www.espressif.com/en/products/socs/esp32-c3

[24] "A vision-based framework for real-time garment piece counting and sewing cycle-time measurement," research manuscript submitted to ICAC, project R26-IT-054, 2026. Unpublished manuscript.

[25] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," *ACM Trans. Graphics*, vol. 23, no. 3, pp. 309–314, 2004.

[26] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.

[27] International Organization for Standardization, "Size designation of clothes — Part 1: Anthropometric definitions for body measurement," ISO 8559-1:2017, 2017.

[28] PyInstaller Development Team, "PyInstaller manual," [Online]. Available: https://pyinstaller.org/

[29] S. Ramírez, "FastAPI," FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[30] SQLite, "Write-ahead logging," SQLite Documentation. [Online]. Available: https://www.sqlite.org/wal.html

[31] Web Bluetooth Community Group, "Web Bluetooth," Draft Community Group Report. [Online]. Available: https://webbluetoothcg.github.io/web-bluetooth/

[32] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[33] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," 2022.

APPENDICES

The appendices provide supplementary material referred to in the main text.

Appendix A Integrated interface specification

Table A.1 specifies how the platform shell starts, configures and reaches each component, and Table A.2 lists the IPC channels exposed to the renderer through the preload bridge. Every IPC handler first checks that the message comes from the application's own window.

Table A.1: Service configuration and endpoints of each component

Component	Start and configuration	Health and readiness	Endpoints reachable from the interface
C1 Production Monitoring	BackendManager; GARMENT_COUNTER_PORT, GARMENT_COUNTER_AUTH_TOKEN, GARMENT_COUNTER_DATA_DIR, GARMENT_COUNTER_MODEL_DIR, GARMENT_COUNTER_ENVIRONMENT	GET /api/health with bearer token; GET /api/readiness	All /api routes through backend:request with token added by the main process, for example /api/sessions, /api/employees, /api/device-configuration, /api/iot-events, /api/vision/start, /api/analytics; Excel export through analytics:export-excel; video through garmentstream://live/session/{id}.mjpeg
C2 Rework Counter	FeatureBackendManager; GARMENT_FEATURE_PORT, GARMENT_REWORK_MODEL_PATH, GARMENT_REWORK_DATA_DIR	GET /health	GET /metrics, /sessions; POST /start, /stop, /update_settings; video through garmentrework://live/video.mjpeg
C3 Style and Colour	StyleColourBackendManager; THREADSCAN_MODEL_PATH, THREADSCAN_DATA_DIR, THREADSCAN_API_PORT (8000), THREADSCAN_VISION_PORT (5050), THREADSCAN_BACKEND_API_BASE	GET / on 8000; GET /api/pipeline/status on 5050	Direct HTTP: /api/garments, /api/settings, /api/device/camera, /api/downtime, /api/stats/daily, /api/stats/summary, /api/predict/next-day on 8000; video feed, pipeline status and camera control on 5050
C4 Size Detection	FeatureBackendManager; GARMENT_FEATURE_PORT, GARMENT_MODEL_PATH, THREADSCAN_DATA_DIR	GET /health	GET /health, /counts, /measurements, /orders; allow listed order and count changes; POST /measure (frame up to 15 MB at IPC, 10 MB at the service)

Table A.2: IPC channels exposed to the renderer

Group	Channels
Platform	desktop:app-info, desktop:readiness, window:minimize, window:toggle-maximize, window:close
C1	backend:status, backend:request, analytics:export-excel, vision:select-validation-video, vision:live-stream-url, iot:discovered-devices, iot:select-device, iot:cancel-device-selection, iot:connection-state, iot:hardware-event
C2	rework:backend-start, rework:backend-status, rework:backend-request, rework:stop-camera, rework:stream-url, rework:select-workflow-video
C3	style-colour:backend-start, style-colour:backend-status, style-colour:stop-camera
C4	size:backend-start, size:backend-status, size:backend-get, size:backend-request, size:backend-measure

Appendix B Team contribution declaration

Each member owned one research component end to end, as summarised in Table 4.5, and contributed to the shared platform and to this report. The shared contributions in Table 4.5 are drawn from the repository history summarised in Section 5.4 and Appendix D. By signing, each member confirms that the record of individual and shared contributions in this report is accurate.

Table B.1: Team contribution declaration

Member	Student ID	Component	Signature	Date
Dharmarathna P.W.A.N.	IT21255892	C1 Production Monitoring		
Pathiraja H.P.M.H.D.S.	IT22627582	C2 Rework Counter		
Jayaweera M.P.Y.C.	IT22556370	C3 Garment Style and Colour Detection		
Jayarathna W.G.U.C.G.	IT22187796	C4 Garment Size Detection		

Appendix C Academic integrity and AI use disclosure

This appendix discloses how generative AI was used in preparing this common report, in accordance with the module policy on academic integrity. The research itself, namely the problem definition, the data collection and annotation, the training of the four models, the design and implementation of each component and of the integrated platform, and the screenshots and project management evidence, was carried out by the members of the group. The four individual theses on which this report is based were prepared by their respective authors and disclose their own use of AI tools separately.

AI was not used to generate data, results or experimental evidence. Every model metric, measurement error, test count and date in this report is taken from the individual theses, the repository, the test outputs or the screenshots supplied by the group, and each source is cited where it is used. Where the sources disagreed, for example between the training record embedded in the deployed size model and the training run reported in the Component 4 thesis, the group decided which evidence to report. The group has reviewed the complete report, checked its statements, figures and references against these sources, and corrected the draft where necessary. The

members remain fully responsible for the accuracy, originality, citations, code, data, analyses and conclusions presented in this report.

Appendix D Repository commit graph

Figures D.1 to D.5 reproduce the commit graph of the shared repository yasasArt/R-26-IT-054, from the first commit on 4 May 2026 to the final merge on 10 October 2026, as shown in the Git Graph view of Visual Studio Code. Each row gives the commit message, date, author and commit identifier, and the coloured lanes show the branches of each member, such as asitha-dev and feature2-garment-counting for C1, Uchith_dev and feature3-size-measurement for C4, and yasas, yasas-Dev and feature4-style-and-colour-detection for C3, and the pull requests through which they were merged into main.

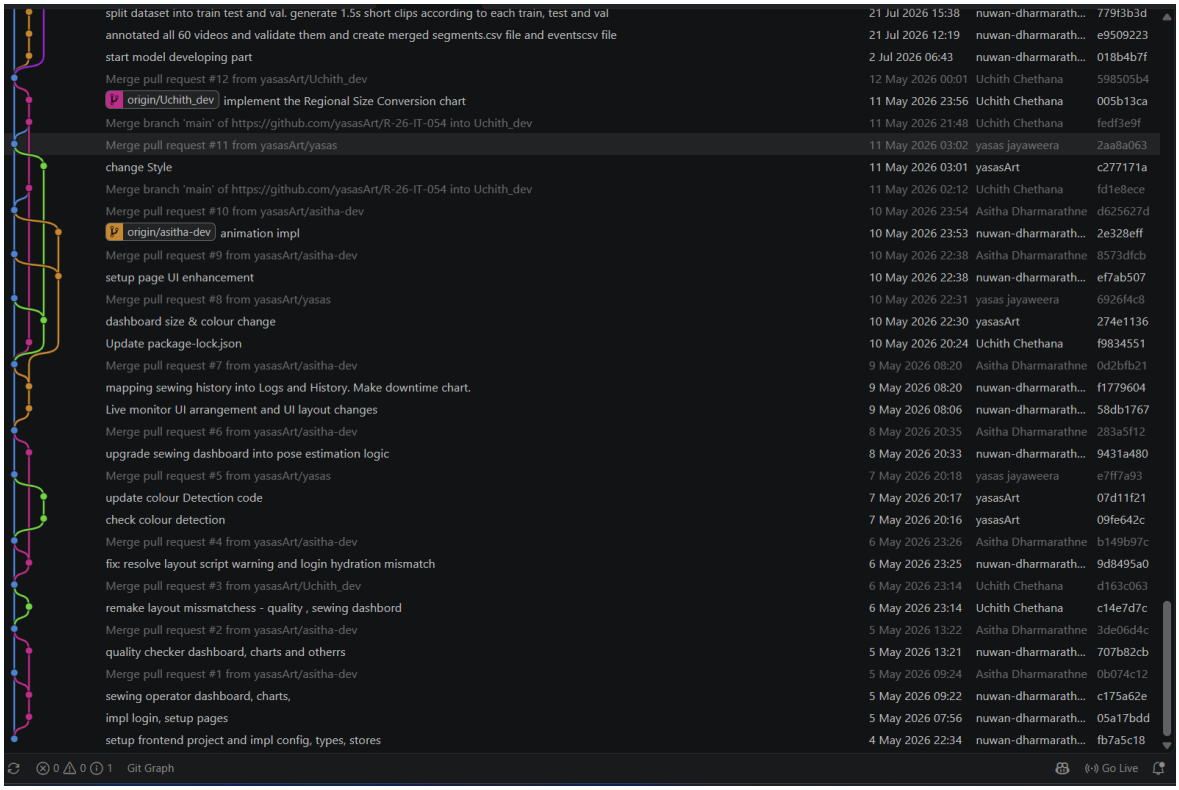


Figure D.1: Commit graph, 4 May to 21 July 2026: shared frontend, first screens and C1 dataset preparation

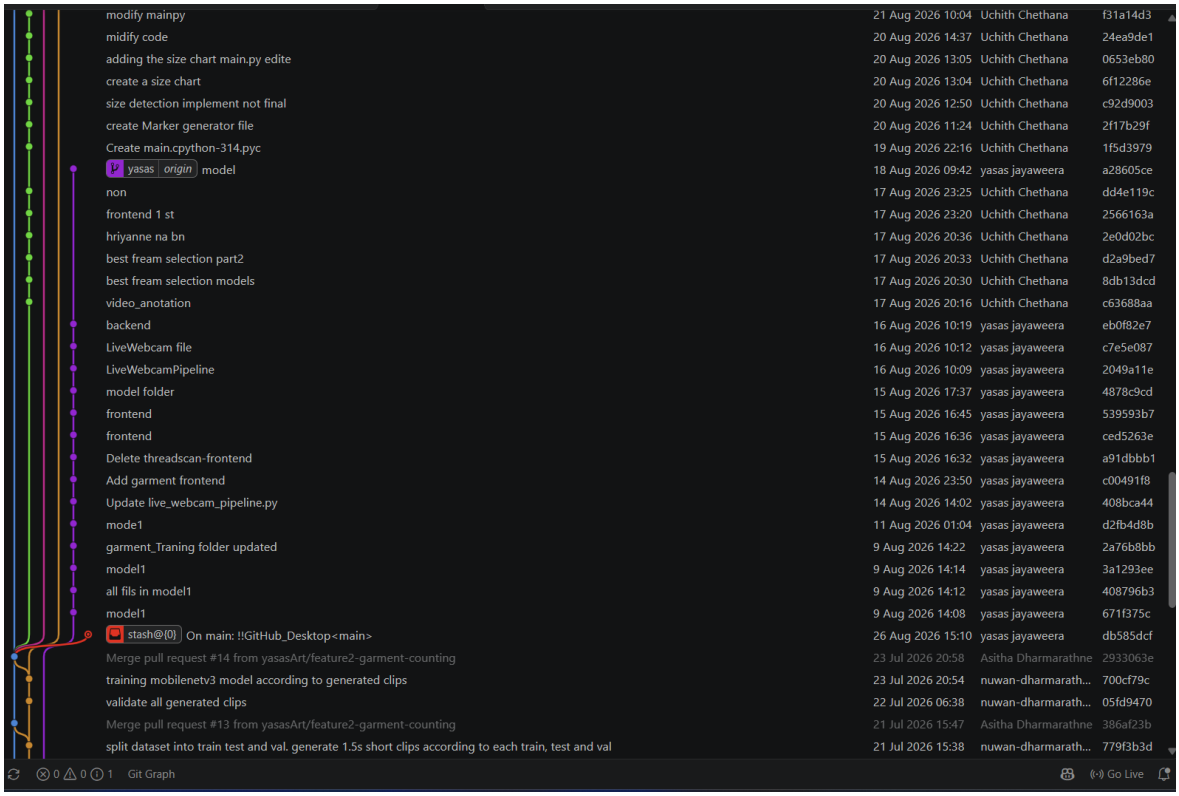


Figure D.2: Commit graph, 21 July to 21 August 2026: C1 classifier training, C3 model and pipeline, C4 best frame selection and size chart

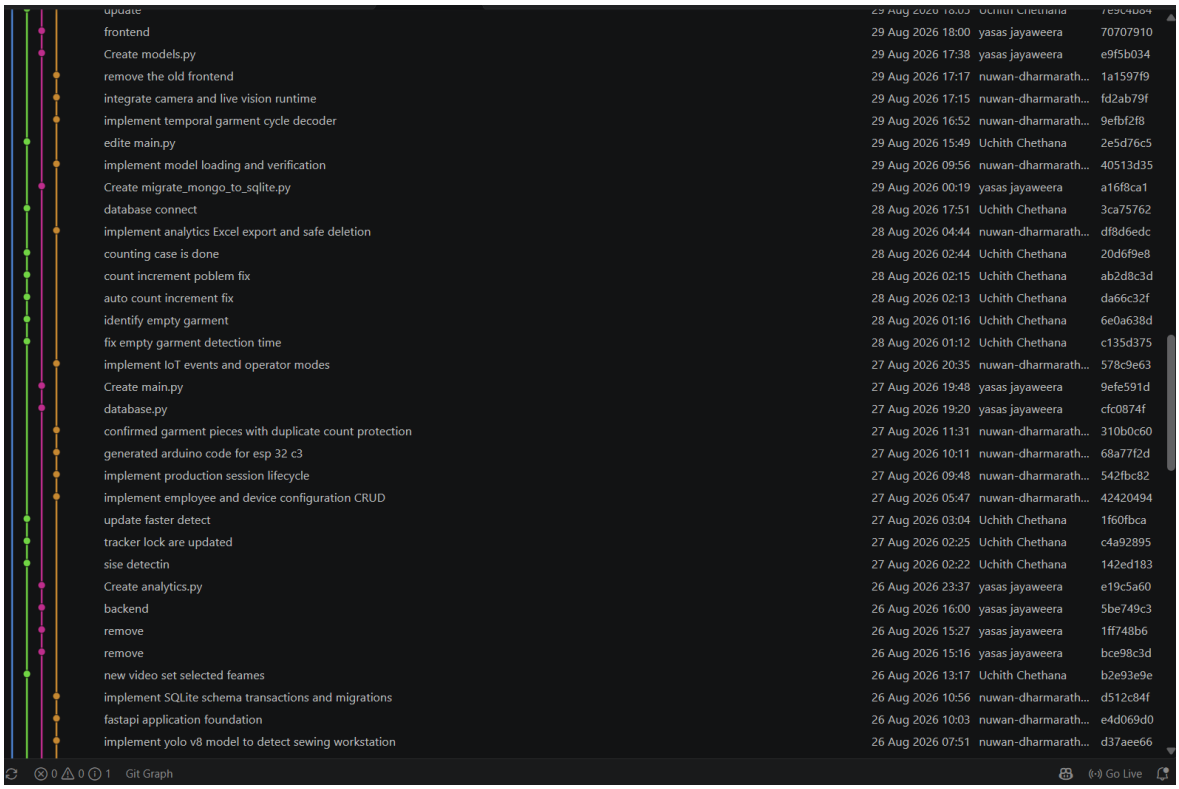


Figure D.3: Commit graph, 26 to 29 August 2026: C1 service, controller and decoder, C3 data service and C4 counting

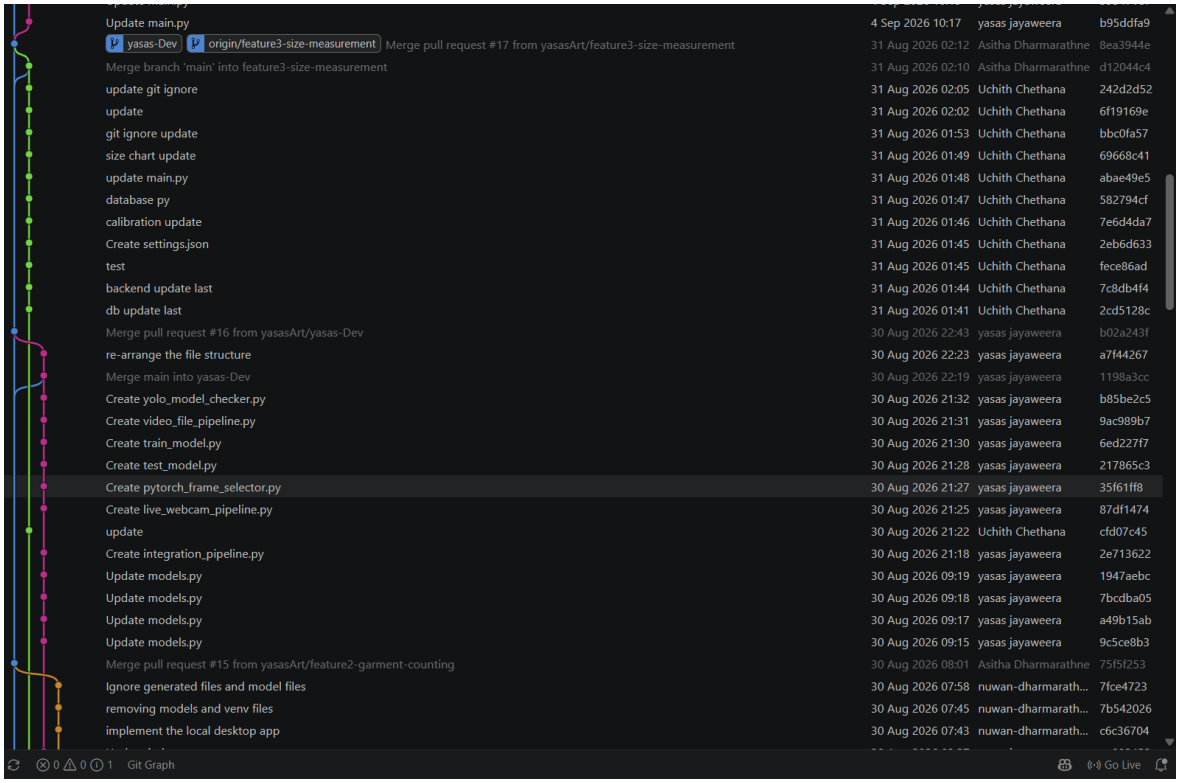


Figure D.4: Commit graph, 30 August to 4 September 2026: local desktop application and size feature merge

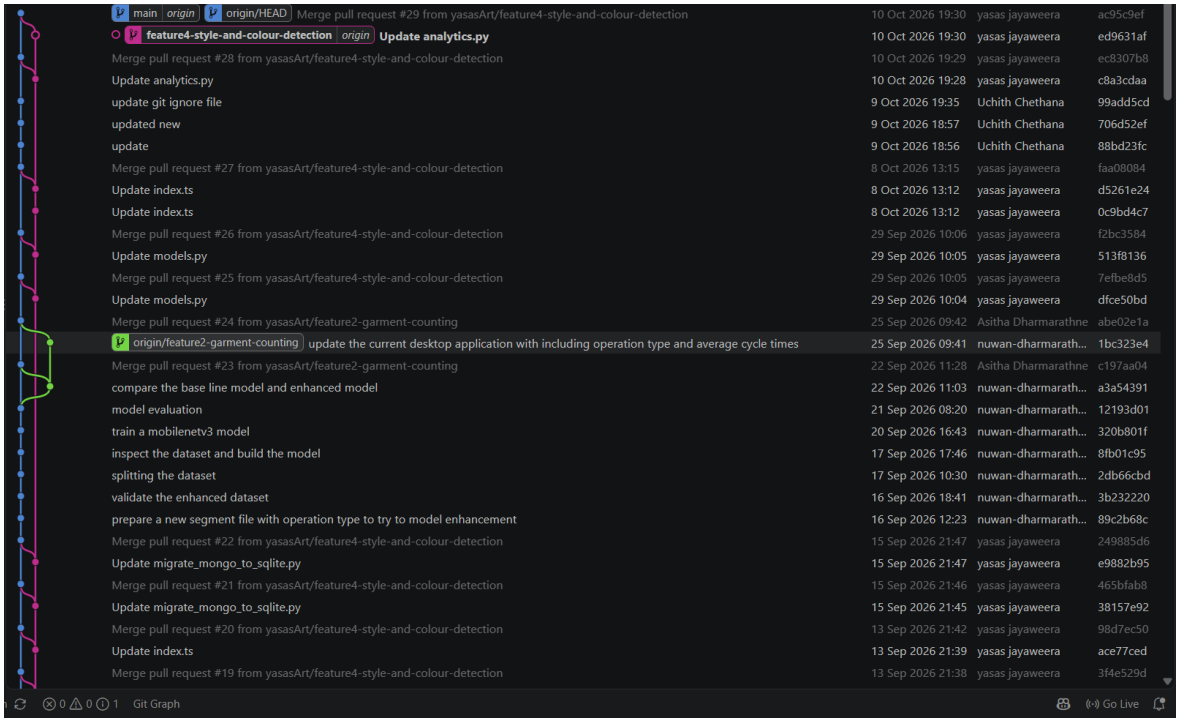


Figure D.5: Commit graph, 13 September to 10 October 2026: C1 operation type model and final integration merges